KTÜ OF TEKNOLOJI FAKÜLTESI · YAZILIM MÜHENDISLIĞI BÖLÜMÜ

Nesne Yönelimli Programlama

2026–2027 Güz Dönemi · Öğrenci Ders Notu

HAFTA 05

Yapıcı ve Yıkıcı Yöntemler

Aşırı yükleme · `this(...)` zinciri · kopya yapıcı · statik fabrika · başlatma sırası · Java'da yıkıcı neden yoktur?

ÖZET

Hafta 4'te nesnenin **yaşarken** geçerli kalmasını sağladık: alanları `private` yaptık, kuralı setter'ın içine taşıdık. Geriye tek bir boşluk kaldı — nesnenin **doğduğu an**. Bu hafta o anı bütünüyle denetim altına alıyoruz. Aşırı yüklenmiş yapıcıları `this(...)` zinciriyle tek bir doğrulama noktasında birleştirecek, kopya yapıcı yazacak, sığ ve derin kopya farkını göreceğiz. Statik fabrika metoduyla nesne üretiminin kendisini kapsülleyecek, bir nesnenin doğuşundaki adım sırasını çalışan kod üzerinde izleyeceğiz. Son bölümde Java'da **yıkıcı (destructor) bulunmamasının** nedenini, çöp toplayıcının ne yapıp ne yapmadığını ve dış kaynakların `AutoCloseable` + `try-with-resources` ile nasıl güvenle kapatıldığını inceleyeceğiz.

Ders	Nesne Yönelimli Programlama (Zorunlu)
Program / Sınıf	Yazılım Mühendisliği · 2. Sınıf · Güz
Haftalık Yük	2 saat teori + 2 saat laboratuvar (toplam 4 saat)
Öğretim Elemanı	Dr. Öğr. Üyesi Rıfat BENVENİSTE
Laboratuvar Sorumlusu	Arş. Gör. Betül MUMCU
Programlama Dili	Java (JDK 21 LTS)
Geliştirme Ortamı	Visual Studio Code + Extension Pack for Java

İçindekiler

1. Bu Haftanın Yol Haritası ... 3

1.1. Ön Koşul Bilgi Kontrolü ... 3

1.2. Haftanın Zaman Planı ... 4

2. Motivasyon: Aynı Sınıf, Beş Farklı Kuruluş Yolu ... 5

2.1. Sorunu Üreten Kod ... 5

2.2. Aynı Sınıf, `this(...)` Zinciriyle ... 6

2.3. Kazancın Ölçüsü ... 7

3. Yapıcı: Tanım, Sözdizimi ve Nesnenin Doğuşu ... 9

3.1. `new` İfadesi Ne Yapar? ... 9

3.2. Yapıcının Biçimsel Kuralları ... 10

3.3. Aşırı Yükleme: İmza Kuralı ... 10

3.4. `this(...)` Zinciri: Sözdizimi ve Çalışma Sırası ... 11

3.5. Yapıcının Doğrulama Yapması ile Setter'ın Doğrulama Yapması ... 12

4. Derinleşme ... 14

4.1. Kopya Yapıcı ... 14

4.2. Sığ Kopya ve Derin Kopya ... 15

4.3. Statik Fabrika Metodu ... 17

4.4. Nesne Başlatma Sırası ... 19

4.5. Yıkıcı Metot: Java'da Neden Yoktur? ... 22

4.6. `AutoCloseable` ve `try-with-resources` ... 24

4.7. Hangi Üretim Yolunu Seçmeli? ... 27

5. C'den Java'ya Köprü ... 29

6. Sık Yapılan Hatalar ... 30

6.1. Yapıcıya Dönüş Türü Yazmak ... 30

6.2. Varsayılan Yapıcının Kaybolduğunu Fark Etmemek ... 30

6.3. `this(...)` Satırını İlk Satır Yapmamak ... 31

6.4. Kopya Yapıcı Yazıp Derinleştirmemek ... 31

6.5. Yapıcıda Aşırı İş Yapmak ... 31

6.6. Kaynağı `try-with-resources` Dışında Açmak ... 31

6.7. Derleyici İletileri Sözlüğü ... 32

7. İyi Tasarım İlkeleri ... 33

7.1. Adlandırma ve Yazım Kuralları ... 33

8. Bir Sonraki Haftaya Köprü ... 34

9. Haftanın Özeti ... 34

10. Alıştırmalar ... 35

10.1. Kavram Soruları ... 35

10.2. Kod Okuma ve Hata Bulma ... 35

10.3. Programlama Alıştırmaları ... 37

11. Öz Değerlendirme 39

Öz Değerlendirme Kontrol Listesi 39

12. Kaynaklar 39

12.1. Ders Kitapları 39

12.2. Çevrimiçi Kaynaklar 40

12.3. İletişim ve Destek 40

1. Bu Haftanın Yol Haritası

Hafta 2'de yapıcı metotla ilk kez tanışmıştık: nesnenin geçerli bir durumda doğmasını sağlayan özel metot. Hafta 4'te ise kapsüllemeyi öğrenip alanları `private` yaptık ve doğrulama kuralını setter'ların içine taşıdık. Bu iki hafta arasında sessizce büyüyen bir gerilim var.

Gerilim şudur: bir sınıfın **birden çok kuruluş yolu** olabilir. Bir öğrenci yalnızca numara ve adla kaydedilebilir; yaşı da bilinebilir; geçmiş notları da devredilebilir. Her yol için ayrı bir yapıcı yazmak doğaldır. Ancak her yapıcıya aynı doğrulama kurallarını kopyalamak, Hafta 4'te öğrendiğimiz "kural tek yerde yazılır" ilkesinin doğrudan ihlalidir.

Bu haftanın ana teknik aracı `this(...)` çağrısıdır: bir yapıcının başka bir yapıcıyı çağırmasını sağlar. Bu araç sayesinde kaç yapıcı yazarsak yazalım, doğrulama kuralı **tek bir ana yapıcıda** toplanır. Haftanın ikinci yarısında ise madalyonun öbür yüzüne bakacağız: nesne öldüğünde ne olur? C++ bilenlerin aradığı **yıkıcı metodun** Java'da neden bulunmadığını ve yerine ne konduğunu göreceğiz.

> **◎ Bu Bölümün Sonunda Öğrenci**
>
> - Bir sınıfa birden çok yapıcı yazabilir; **aşırı yükleme** kurallarını (imza farkı) doğru uygulayabilir.
> - `this(...)` çağrısıyla yapıcıları zincirleyebilir; doğrulama kuralını tek bir ana yapıcıda toplayabilir.
> - `this(...)` çağrısının **ilk satır olma** zorunluluğunu ve gövdelerin hangi sırayla çalıştığını çıktı üzerinde gösterebilir.
> - Kopya yapıcı yazabilir; **sığ (shallow)** ve **derin (deep)** kopya arasındaki farkı somut bir çıktıyla açıklayabilir.
> - Statik fabrika metodu tasarlayabilir; bu desenin yapıcıya göre dört üstünlüğünü ve bedelini sayabilir.
> - Bir nesne doğarken static bölüm, örnek alan başlatıcıları, örnek blokları ve yapıcı gövdesinin hangi sırayla çalıştığını **doğru sırayla** yazabilir.
> - Java'da yıkıcı (destructor) bulunmamasının nedenini açıklayabilir; `finalize()` metodunun neden kullanılmadığını gerekçelendirebilir.
> - `AutoCloseable` arayüzünü gerçekleyen bir sınıf yazabilir ve onu `try-with-resources` bloğuyla kullanarak kaynak sızıntısını önleyebilir.

1.1. Ön Koşul Bilgi Kontrolü

Aşağıdaki soruların tamamını Hafta 1–4 bilgisiyle yanıtlayabiliyor olmalısınız. Yanıtlayamadığınız madde varsa laboratuvar öncesinde ilgili bölümü tekrar etmeniz önerilir.

1. Yapıcı metodu sıradan bir metottan ayıran **iki** biçimsel özellik nedir? (Hafta 2, Bölüm 3)
2. `this.ad = ad;` satırında `this` neyi işaret eder? Bu satır `this` olmadan yazılsaydı ne olurdu?
3. Metot aşırı yükleme (overloading) nedir? İki metodun aynı ada sahip olabilmesi için hangi koşul sağlanmalıdır?
4. Hafta 4'te doğrulama kuralını neden setter'ın içine taşımıştık? Aynı kuralı yapıcıda ikinci kez yazmaktan nasıl kaçınmıştık?

5. `final` bir alan ne zaman ve kaç kez atanabilir? `private final int[] notlar;` bildirimi dizinin içeriğini korur mu?
6. Referans türü bir değişkeni başka bir değişkene atadığımızda kaç nesne olur? `a == b` karşılaştırması neyi ölçer?

1.2. Haftanın Zaman Planı

Oturum	Süre	İçerik	Çıktı
Teori 1	45 dk	Motivasyon · yapıcı aşırı yükleme · `this(...)` zinciri · kopya yapıcı · sığ/derin kopya	Tek doğrulama noktalı çoklu yapıcı yazabilme
Teori 2	45 dk	Statik fabrika metodu · nesne başlatma sırası · Java'da yıkıcı yokluğu · GC · `AutoCloseable`	Kaynak kapatmayı garantileyebilme
Lab 1	60 dk	Görev 0–2: ortam kontrolü · çoklu yapıcı zinciri · kopya yapıcı ve derin kopya	Çalışan ve doğrulayan iki sınıf
Lab 2	60 dk	Görev 3–5: statik fabrika · `AutoCloseable` · (bonus) başlatma sırası deneyi	Teslim edilecek `.zip` paketi

Tablo 5.1 — Hafta 5'in oturum planı ve beklenen çıktılar

> **ⓘ BİLGİ**
>
> Bu hafta öğrenilecek yeni anahtar sözcük sayısı yine azdır: `this(...)` çağrı biçimi, `static` başlatma bloğu ve `AutoCloseable` arayüzü. Zorluk, Hafta 4'te olduğu gibi **karar vermededir**: kaç yapıcı yazılacağına, hangisinin ana yapıcı olacağına ve nesne üretiminin yapıcıyla mı fabrika metoduyla mı yapılacağına karar vermek bu haftanın asıl becerisidir.

2. Motivasyon: Aynı Sınıf, Beş Farklı Kuruluş Yolu

Bir kütüphane yazılımı düşünelim. `Kitap` nesneleri sisteme üç ayrı kanaldan giriyor: (i) barkod okuyucudan yalnızca ISBN ve başlık geliyor, (ii) yayınevi listesinden ISBN, başlık ve yazar geliyor, (iii) elle girişte tüm alanlar doldurulyor. Üç kanal için üç ayrı yapıcı yazmak son derece doğaldır.

2.1. Sorunu Üreten Kod

Kuralı her yapıcıya kopyalayan ilk yaklaşım şöyle görünür:

Kitap.java (sorunlu sürüm)

```
// SORUNLU TASARIM - dogrulama kurali UC KEZ yazilmis
public class Kitap {
    private final String isbn;
    private String baslik;
    private String yazar;
    private int sayfaSayisi;

    public Kitap(String isbn, String baslik) {
        if (isbn == null || isbn.isBlank()) throw new IllegalArgumentException("...");
        if (baslik == null || baslik.isBlank()) throw new IllegalArgumentException("...");
        this.isbn = isbn; this.baslik = baslik;
        this.yazar = "Bilinmiyor"; this.sayfaSayisi = 100;
    }

    public Kitap(String isbn, String baslik, String yazar) {
        if (isbn == null || isbn.isBlank()) throw new IllegalArgumentException("...");
        if (baslik == null || baslik.isBlank()) throw new IllegalArgumentException("...");
        this.isbn = isbn; this.baslik = baslik;
        this.yazar = yazar; this.sayfaSayisi = 100;
    }

    public Kitap(String isbn, String baslik, String yazar, int sayfaSayisi) {
        if (isbn == null || isbn.isBlank()) throw new IllegalArgumentException("...");
        if (baslik == null || baslik.isBlank()) throw new IllegalArgumentException("...");
        // sayfa sayisi denetimi burada UNUTULDU
        this.isbn = isbn; this.baslik = baslik;
        this.yazar = yazar; this.sayfaSayisi = sayfaSayisi;
    }
}
```

Kod 5.1 — Doğrulamayı her yapıcıya kopyalayan tasarım

Bu kodda üç ayrı kusur vardır ve üçü de aynı kökten çıkar:

1. **Kural üç kez yazılmıştır.** ISBN denetimi değişirse üç yerde değiştirilmelidir; biri unutulursa sınıf kendi kuralını çiğnemeye başlar.
2. **Üçüncü yapıcıda `sayfaSayisi` denetimi zaten unutulmuştur.** Bu, kopyala-yapıştır tasarımının kaçınılmaz sonucudur: gözden kaçan bir satır sessizce geçersiz nesne üretir.
3. **Varsayılan değerler (`"Bilinmiyor"`, `100`) iki yerde tekrarlanır.** Varsayılan değişirse yine iki dosyada değil, iki yapıcıda arama yapmak gerekir.

> ✕ **SIK YAPILAN HATA**
>
> Bu, Hafta 4'te tanıştığımız *gecikmiş hata* deseninin yapıcı sürümüdür. Hata, üçüncü yapıcının yazıldığı gün oluşur; ancak haftalar sonra "sayfa sayısı sıfır olan kitap" raporda göründüğünde fark edilir. O noktada hatanın kaynağını bulmak, ilk günkü maliyetin katbekat üstündedir.

2.2. Aynı Sınıf, `this(...)` Zinciriyle

Şimdi aynı dört yapıcıyı, doğrulamayı **tek bir yerde** toplayarak yazalım. Kısa yapıcılar eksik bilgiyi varsayılanla tamamlar ve işi bir sonrakine devreder; kural yalnızca zincirin ucundaki **ana yapıcıda** bulunur.

● *Kitap.java*

```java
    public Kitap(String isbn, String baslik) {
        this(isbn, baslik, BILINMIYOR);
    }

    /* --- 2. Yazar da verilir --- */
    public Kitap(String isbn, String baslik, String yazar) {
        this(isbn, baslik, yazar, 100);
    }

    /* --- 3. Sayfa sayisi da verilir --- */
    public Kitap(String isbn, String baslik, String yazar, int sayfaSayisi) {
        this(isbn, baslik, yazar, sayfaSayisi, 2026);
    }
```

Kod 5.2 — Üç kısa yapıcı: her biri bir eksiği tamamlar ve devreder

● *Kitap.java*

```java
    /* --- 4. ANA YAPICI: tum dogrulama burada --- */
    public Kitap(String isbn, String baslik, String yazar, int sayfaSayisi, int basimYili) {
        if (isbn == null || isbn.isBlank()) {
            throw new IllegalArgumentException("ISBN bos olamaz");
        }
        if (baslik == null || baslik.isBlank()) {
            throw new IllegalArgumentException("Baslik bos olamaz");
        }
        if (sayfaSayisi < EN_AZ_SAYFA || sayfaSayisi > EN_COK_SAYFA) {
            throw new IllegalArgumentException("Sayfa sayisi gecersiz: " + sayfaSayisi);
        }
        if (basimYili < 1450 || basimYili > 2026) {
            throw new IllegalArgumentException("Basim yili gecersiz: " + basimYili);
        }
        this.isbn = isbn.trim();
        this.baslik = baslik.trim();
        this.yazar = (yazar == null || yazar.isBlank()) ? BILINMIYOR : yazar.trim();
        this.sayfaSayisi = sayfaSayisi;
        this.basimYili = basimYili;
    }
```

Kod 5.3 — Ana yapıcı: dört kuralın tamamı burada, tek kopya

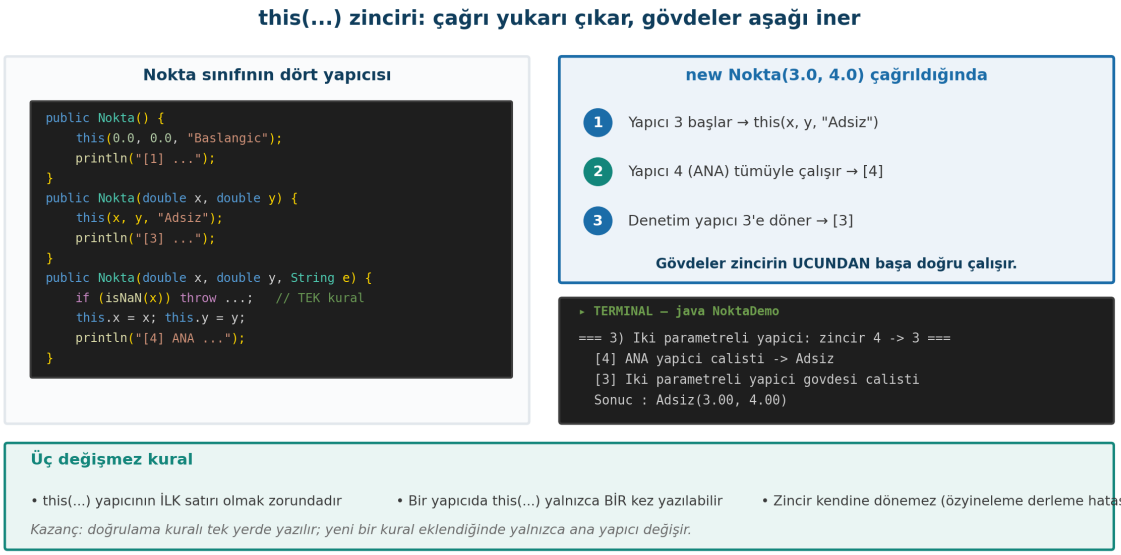
```
▸ TERMINAL — java Gorev1_KitapDemo (ilk bölüm)
=== 1) Dort yapici, dort ayri eksiklik duzeyi ===
  Kitap[978-975-01, "Nesne Yonelimli Programlama", Bilinmiyor, 100 s., 2026]
  Kitap[978-975-02, "Veri Yapilari", H. Yilmaz, 100 s., 2026]
  Kitap[978-975-03, "Algoritmalar", S. Demir, 720 s., 2026]
  Kitap[978-975-04, "Isletim Sistemleri", A. Kara, 640 s., 2019]

=== 2) Varsayilanlarin nereden geldigi ===
  k1 yazar       : Bilinmiyor   (2. yapicidan)
  k1 sayfa       : 100          (3. yapicidan)
  k1 basim yili  : 2026         (4. yapicidan)
```

Dört farklı çağrı biçimi, dört farklı nesne üretti; ancak doğrulama yalnızca bir kez yazıldı. Üstelik varsayılan değerler de zincirin kendisinde tanımlıdır: `"Bilinmiyor"` ikinci yapıcıda, `100` üçüncüde, `2026` dördüncüde. Bir varsayılanı değiştirmek için değiştirilecek satır sayısı **birdir**.

```
▸ TERMINAL — java Gorev1_KitapDemo (doğrulama)
=== 3) Dogrulama TEK yerde: ana yapici ===
  RET      : Sayfa sayisi gecersiz: 0
  RET      : ISBN bos olamaz
  RET      : Baslik bos olamaz
  RET      : Basim yili gecersiz: 2030
```

Dikkat ediniz: dört ret iletisi de **ana yapıcıdan** gelmektedir. `new Kitap("", "Adsiz Kitap", "M. Ak", 300, 2020)` çağrısı beş parametreliydi, `new Kitap("978-975-05", ...)` çağrısı ise zinciri kullandı; ikisi de aynı kural kümesine çarptı.



Şekil 5.1 — `this(...)` zinciri: çağrı yukarı çıkar, gövdeler aşağı iner

2.3. Kazancın Ölçüsü

Ölçüt	Kural her yapıcıda	`this(...)` zinciriyle

Ölçüt	Kural her yapıcıda	`this(...)` zinciriyle
Doğrulama kuralının yazıldığı yer	N yapıcı	1 yapıcı
Kural değişince dokunulacak satır	N × kural sayısı	kural sayısı
Bir yapıcıda kuralı unutma riski	Yüksek	Yok
Varsayılan değerin tanımlandığı yer	Her kısa yapıcıda	Zincirde tek
Yeni yapıcı eklemenin maliyeti	Tüm kuralları kopyalamak	Tek satır `this(...)`
Kod satır sayısı (4 yapıcı, 4 kural)	~48 satır	~22 satır
Okunabilirlik	Hangi yapıcı ne yapıyor belirsiz	Zincir, eksiği açıkça gösterir

Tablo 5.2 — Doğrulamayı tek noktada toplamanın somut sonuçları

> ✓ **İPUCU**
>
> Zincirin yönü de bilgi taşır: **kısa yapıcıdan uzun yapıcıya** doğru gidilir. Her adımda bir parametre eklenir. Bu düzen okuyucuya “hangi bilgi zorunlu, hangisi isteğe bağlı” sorusunun yanıtını doğrudan verir. Ters yön (uzundan kısaya) hem anlamsızdır hem de varsayılanları saklanamaz hâle getirir.

3. Yapıcı: Tanım, Sözdizimi ve Nesnenin Doğuşu

Yapıcı metot (constructor) Bir nesne `new` ile üretildiğinde otomatik olarak çalışan, adı sınıf adıyla birebir aynı olan ve **dönüş türü bulunmayan** özel metot. Görevi, hâlihazırda ayrılmış belleği **geçerli bir başlangıç durumuna** getirmektir.

Yapıcı aşırı yükleme (constructor overloading) Aynı sınıfta, parametre listeleri birbirinden farklı birden çok yapıcı tanımlamak. Ayrım yalnızca **imzaya** (parametre sayısı, türü ve sırası) dayanır; dönüş türü söz konusu değildir.

`this(...)` çağrısı Bir yapıcının aynı sınıftaki başka bir yapıcıyı çağırması. Yapıcının **ilk satırı** olmak zorundadır ve bir yapıcıda en fazla bir kez yazılabilir.

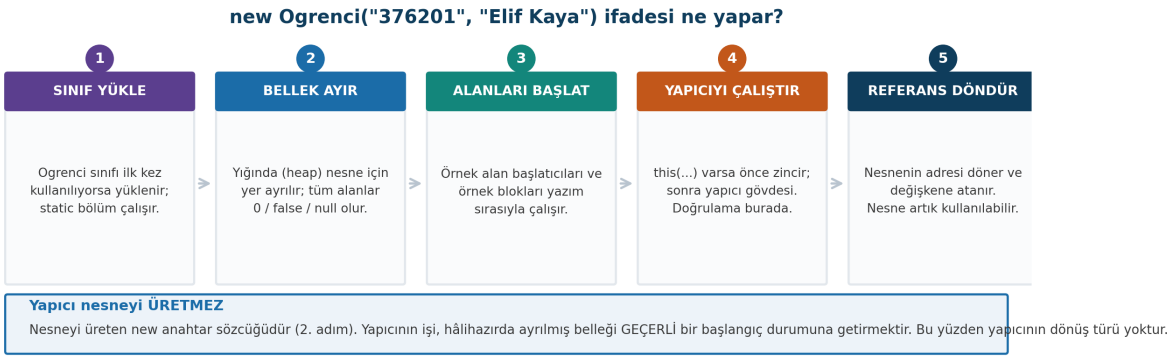
Ana yapıcı (designated / telescoping constructor) Zincirin ucundaki, tüm parametreleri alan ve doğrulamanın tamamını barındıran yapıcı. Diğer tüm yapıcılar doğrudan ya da dolaylı olarak buna devreder.

Kopya yapıcı (copy constructor) Parametre olarak kendi sınıfından bir nesne alan ve onun durumunu taşıyan **yeni** bir nesne üreten yapıcı.

Statik fabrika metodu (static factory method) Nesne üretimini `new` yerine adlandırılmış bir `static` metot üzerinden yapan desen. Yapıcı çoğunlukla `private` yapılır.

3.1. `new` İfadesi Ne Yapar?

Yapıcıyı doğru anlamak için önce `new` anahtar sözcüğünün ne yaptığını netleştirmek gerekir. Yaygın bir yanlış anlama, “yapıcı nesneyi üretir” cümlesidir. Nesneyi üreten `new`'dir; yapıcı, üretilmiş nesneyi **başlatır**.



Şekil 5.2 — `new` ifadesinin beş adımı — yapıcı yalnızca dördüncü adımdır

Bu ayrım iki soruyu doğrudan yanıtlar. Birincisi: yapıcının dönüş türü neden yoktur? Çünkü döndürecek bir şeyi yoktur; referansı `new` döndürür. İkincisi: yapıcının içinde `this` neden zaten kullanılabilir? Çünkü nesne yapıcı çalışmaya başlamadan **önce** bellekte vardır.

3.2. Yapıcının Biçimsel Kuralları

Özellik	Yapıcı	Sıradan metot

Özellik	Yapıcı	Sıradan metot
Ad	Sınıf adıyla **birebir aynı**	Serbest (camelCase)
Dönüş türü	**Yoktur** — `void` bile yazılmaz	Zorunlu
Çağrılma biçimi	Yalnızca `new` ile (ya da `this(...)`)	`nesne.metot()`
Kalıtımla geçer mi?	Hayır (Hafta 7)	Evet
`static` olabilir mi?	Hayır	Evet
Aşırı yüklenebilir mi?	Evet	Evet
Kaç kez çalışır?	Nesne başına tam bir kez	İstendiği kadar
Yazılmazsa ne olur?	Derleyici parametresiz yapıcı **ekler**	Hiçbir şey

Tablo 5.3 — Yapıcı ile sıradan metodun karşılaştırması

> **▲ DİKKAT**
>
> Son satır Hafta 2'de gördüğümüz tuzağın hatırlatmasıdır: derleyicinin eklediği parametresiz yapıcı, **siz herhangi bir yapıcı yazar yazmaz kaybolur**. `Kitap(String, String)` yapıcısını yazdıktan sonra `new Kitap()` çağrısı derleme hatası verir. Parametresiz kuruluş hâlâ isteniyorsa açıkça yazılmalıdır.

3.3. Aşırı Yükleme: İmza Kuralı

İki yapıcının birlikte var olabilmesi için **imzalarının** farklı olması gerekir. İmza, parametrelerin sayısı, türü ve sırasından oluşur. Parametre **adları** imzanın parçası değildir.

1. yapıcı	2. yapıcı	Derlenir mi?	Neden?
`Bilet(String k, String v)`	`Bilet(String a, String b)`	**Hayır**	Adlar farklı ama imza aynı
`Nokta(double x)`	`Nokta(double x, double y)`	Evet	Parametre **sayısı** farklı
`Nokta(double x, double y)`	`Nokta(int x, int y)`	Evet	Parametre **türü** farklı
`Kayit(String no, int kredi)`	`Kayit(int kredi, String no)`	Evet	Parametre **sırası** farklı
`Kitap(String isbn)`	`Kitap(String baslik)`	**Hayır**	İkisi de tek `String`; anlam farkı imzaya yansımaz

Tablo 5.4 — Yapıcı aşırı yükleme kuralının uygulanışı

> **✕ İMZA KISITI — yapıcının doğal sınırı**
>
> Tablonun son satırı bu haftanın en can sıkıcı kısıtıdır: `Bilet` sınıfında "tam bilet", "öğrenci bileti" ve "yaşlı bileti" üçü de `(String kalkis, String varis)` imzasını gerektirir. Yapıcıyla bu **yazılamaz**. Çözümü Bölüm 4.4'teki statik fabrika metodudur.

3.4. `this(...)` Zinciri: Sözdizimi ve Çalışma Sırası

`Nokta` sınıfının dört yapıcısını inceleyelim. Üçü kısa, biri anadır; her kısa yapıcı `this(...)` ile bir üstüne devreder.

Nokta.java

```
1      /* --- 1. Parametresiz yapici: her seyi varsayilana birakir --- */
2      public Nokta() {
3          this(0.0, 0.0, "Baslangic");
4          System.out.println("  [1] Parametresiz yapici govdesi calisti");
5      }
6
7      /* --- 2. Tek parametreli: kosegen uzerinde bir nokta --- */
8      public Nokta(double kosegen) {
9          this(kosegen, kosegen, "Kosegen");
10         System.out.println("  [2] Tek parametreli yapici govdesi calisti");
11     }
12
13     /* --- 3. Iki parametreli: en sik kullanilan bicim --- */
14     public Nokta(double x, double y) {
15         this(x, y, "Adsiz");
16         System.out.println("  [3] Iki parametreli yapici govdesi calisti");
17     }
```

Kod 5.4 — Üç kısa yapıcı: her biri `this(...)` ile devreder

Satır satır çözümleyelim (satır numaraları **yukarıdaki kod listesine** göredir, dosyadaki mutlak numaralara değil):

1. **3.** `this(0.0, 0.0, "Baslangic");` — parametresiz yapıcı, üç varsayılanı belirleyip ana yapıcıya devreder. Bu satır yapıcının **ilk** satırıdır; öncesine hiçbir ifade yazılamaz.
2. **4.** Devir tamamlandıktan sonra bu yapıcının kendi gövdesi çalışır. Buradaki `println` çağrısı, zincirin dönüş yönünü ekranda görebilmek için konmuştur.
3. **9.** Tek parametreli yapıcı, verilen değeri hem `x` hem `y` için kullanır. Bir yapıcının parametreleri **anlamlandırma** hakkı vardır; burada tek sayı "köşegen üzerindeki nokta" olarak yorumlanmıştır.
4. **15.** İki parametreli yapıcı, yalnızca etiketi varsayılana bırakır. En sık kullanılan biçim çoğunlukla budur.

Nokta.java

```
1      /* --- 4. ANA YAPICI: dogrulama yalnizca burada --- */
2      public Nokta(double x, double y, String etiket) {
3          if (Double.isNaN(x) || Double.isNaN(y)) {
4              throw new IllegalArgumentException("Koordinat NaN olamaz");
5          }
6          this.x = x;
7          this.y = y;
8          this.etiket = (etiket == null || etiket.isBlank()) ? "Adsiz" : etiket.trim();
9          System.out.println("  [4] ANA yapici calisti -> " + this.etiket);
10     }
11
12     /* --- 5. KOPYA YAPICI: ayni turden bir nesneden yeni nesne uretir --- */
13     public Nokta(Nokta kaynak) {
14         this(kaynak.x, kaynak.y, kaynak.etiket + " (kopya)");
15     }
```

Kod 5.5 — Ana yapıcı ve kopya yapıcı — doğrulama yalnızca ana yapıcıda

Ana yapıcının 3–5. satırlarındaki denetim, sınıfın **tek** doğrulama noktasıdır. Hangi yapıcı çağrılırsa çağrılsın bu satırlardan geçilir; geçilmeden alan ataması yapılamaz. 14. satırdaki kopya yapıcı da kendini bu kurala tabi kılar: o da `this(...)` ile ana yapıcıya devreder.

```
▸ TERMINAL — java NoktaDemo (zincirin gözlemi)
=== 1) Parametresiz yapici: zincir 4 -> 1 ===
  [4] ANA yapici calisti -> Baslangic
  [1] Parametresiz yapici govdesi calisti
  Sonuc : Baslangic(0.00, 0.00)

=== 2) Tek parametreli yapici: zincir 4 -> 2 ===
  [4] ANA yapici calisti -> Kosegen
  [2] Tek parametreli yapici govdesi calisti
  Sonuc : Kosegen(3.00, 3.00)

=== 3) Iki parametreli yapici: zincir 4 -> 3 ===
  [4] ANA yapici calisti -> Adsiz
  [3] Iki parametreli yapici govdesi calisti
  Sonuc : Adsiz(3.00, 4.00)
  Baslangictan uzaklik : 5.00
```

Çıktının en öğretici yanı **sıradır**. Her üç durumda da `[4] ANA yapici` satırı, kısa yapıcının kendi satırından **önce** yazılmıştır. Bunun nedeni açıktır: `this(...)` bir çağrıdır ve çağrılan yapıcı tamamlanmadan çağıran yapıcının bir sonraki satırına geçilemez. Yani **çağrı zincirde yukarı çıkar, gövdeler aşağı iner**.

> **ⓘ BİLGİ**
>
> Bu davranış, `final` alanlarla birlikte düşünüldüğünde neden `this(...)` çağrısının ilk satır olmak zorunda olduğunu da açıklar. `x`, `y` ve `etiket` alanları `final`'dır; ana yapıcıda bir kez atanırlar. Eğer kısa yapıcı `this(...)`'ten önce bu alanlara dokunabilseydi, `final` alan iki kez atanmış olurdu.

3.5. Yapıcının Doğrulama Yapması ile Setter'ın Doğrulama Yapması

Hafta 4'te geçersiz bir değer geldiğinde setter uyarı yazıp atamayı yapmıyordu; nesne eski geçerli durumunu koruyordu. Yapıcıda ise böyle bir "eski geçerli durum" **yoktur**: nesne henüz doğmamıştır. Bu yüzden yapıcıda doğru davranış, nesnenin hiç doğmamasını sağlamaktır.

Ogrenci.java

```
1       /* --- 3. ANA YAPICI: butun dogrulama burada, tek kopya --- */
2       public Ogrenci(String ogrenciNo, String ad, int yas, int[] notlar) {
3           if (ogrenciNo == null || ogrenciNo.isBlank()) {
4               throw new IllegalArgumentException("Ogrenci numarasi bos olamaz");
5           }
6           if (ad == null || ad.isBlank()) {
7               throw new IllegalArgumentException("Ad bos olamaz");
8           }
```

```
 9          if (yas < EN_KUCUK_YAS || yas > EN_BUYUK_YAS) {
10              throw new IllegalArgumentException(
11                      "Yas " + EN_KUCUK_YAS + ".." + EN_BUYUK_YAS + " araliginda olmali:
" + yas);
12          }
13          this.ogrenciNo = ogrenciNo.trim();
14          this.ad = ad.trim();
15          this.yas = yas;
16          this.notlar = notlar.clone();          // disaridaki dizi ile bagi kes
17          this.notAdedi = 0;
18      }
```

Kod 5.6 — Yapıcıda doğrulama: geçersiz değerde nesne hiç üretilmez

```
▸ TERMINAL — java OgrenciDemo (üç geçersiz istek)
=== 2) Gecersiz istek: nesne HIC dogmuyor ===
  RET      : Ad bos olamaz
  RET      : Yas 17..70 araliginda olmali: 12
  RET      : Ogrenci numarasi bos olamaz
```

Üç istek de reddedildi ve hiçbiri için nesne üretilmedi. `throw` ifadesi metodun çalışmasını o satırda keser; sonraki atama satırlarına hiç ulaşılmaz. Bu, Hafta 4'te sözünü verdiğimiz “gerçek projelerde tercih edilen yol”dur; istisna mekanizmasının tamamını Hafta 11'de ele alacağız.

Konum	Geçersiz değer gelince	Nesnenin durumu	Bu haftaki karşılığı
Setter	Uyarı yazılır, atama yapılmaz	Eski geçerli durum korunur	`Ogrenci.setYas()`
Yapıcı	`IllegalArgumentException` fırlatılır	Nesne **hiç doğmaz**	`Ogrenci` ana yapıcısı
Fabrika metodu	Denetimli bir yedek nesne döndürülür	Geçerli ama “boş” nesne	`Kayit.olustur()` → `Kayit.BOS`

Tablo 5.5 — Geçersiz değerin üç farklı karşılanma biçimi

Satır 50'deki `this.notlar = notlar.clone();` ifadesi bu haftanın en kolay atlanan ayrıntısıdır. Yapıcıya dışarıdan bir dizi verilmiştir; kopyalanmasaydı çağıran kod, nesne doğduktan sonra bile o dizinin içeriğini değiştirebilirdi. Kapsülleme yapıcıda başlar.

```
▸ TERMINAL — java OgrenciDemo (yapıcıda kopyalama)
=== 5) Ana yapicida clone() olmasaydi ne olurdu? ===
  Disaridaki dizi degistirildi: disDizi[0]=999
  Nesne icindeki ilk not : 50
  Nesne etkilenmedi; cunku yapici diziyi KOPYALADI.
```

4. Derinleşme

4.1. Kopya Yapıcı

Bir nesnenin “aynısından bir tane daha” gerektiği durumlar sıktır: kaydı düzenlemeye başlamadan önce yedek almak, bir nesneyi metoda verirken orijinali korumak, bir listeyi çoğaltmak. Bu iş için Java'da kullanılan desen **kopya yapıcıdır**.



Şekil 5.3 — Atama yeni nesne üretmez; kopya yapıcı üretir

Şekildeki yanlış yaklaşım, C bilen öğrencilerin en sık düştüğü tuzaktır. C'de `Ogrenci k = asil;` satırı yapının tüm baytlarını kopyalardı; Java'da ise yalnızca **referans** kopyalanır. Ortada tek nesne, iki ad vardır.

Ogrenci.java

```
1   /* --- 4. KOPYA YAPICI: derin kopya --- */
2   public Ogrenci(Ogrenci kaynak) {
3       if (kaynak == null) {
4           throw new IllegalArgumentException("Kopyalanacak ogrenci null olamaz");
5       }
6       this.ogrenciNo = kaynak.ogrenciNo;
7       this.ad = kaynak.ad;
8       this.yas = kaynak.yas;
9       this.notlar = kaynak.notlar.clone();   // DERIN: yeni dizi uretilir
10      this.notAdedi = kaynak.notAdedi;
11  }
```

Kod 5.7 — `Ogrenci` sınıfının kopya yapıcısı

Listenin 2. satırındaki imza kopya yapıcıyı tanımlar: parametre, sınıfın **kendi türündedir**. 3–4. satırlardaki `null` denetimi, `kaynak.ogrenciNo` ifadesinin `NullPointerException` fırlatmasını önler. 9. satırdaki `clone()` çağrısı ise kopyanın **derin** olmasını sağlar; bunun ne demek olduğunu bir sonraki bölümde göreceğiz.

```
▸ TERMINAL — java OgrenciDemo (kopya yapıcı)
=== 4) KOPYA YAPICI: derin kopya ===
```

```
Kaynak : Ogrenci[376201, Elif Kaya, 18 yas, 3 not, ort=83.33]
Kopya  : Ogrenci[376201, Elif Kaya, 18 yas, 3 not, ort=83.33]
Ayni nesne mi? false
--- kopya degistirildi ---
Kaynak : Ogrenci[376201, Elif Kaya, 18 yas, 3 not, ort=83.33]
Kopya  : Ogrenci[376201, Elif Kaya (2. kayit), 18 yas, 4 not, ort=72.50]
Kaynak etkilenmedi -> DERIN kopya
```

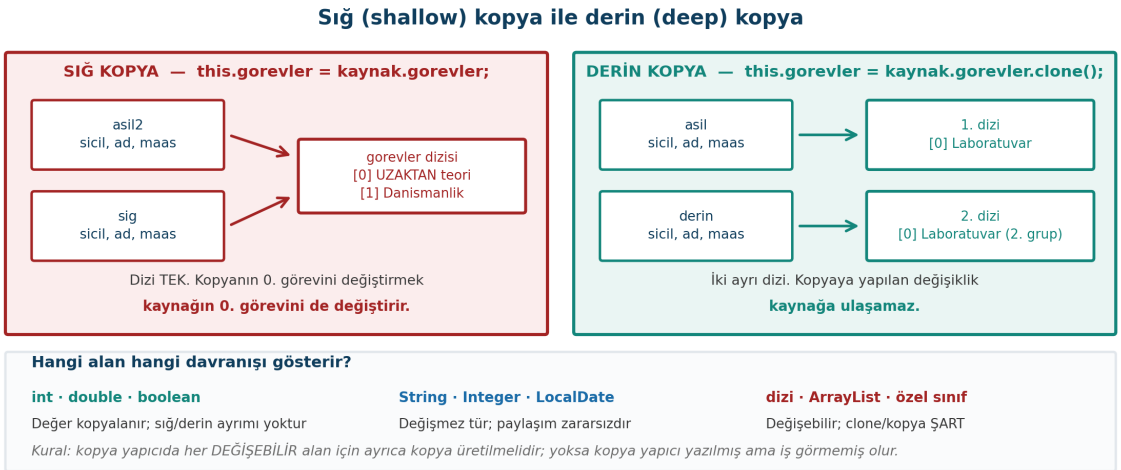
Alan	Türü	Kopya yapıcıda ne yapıldı?	Neden?
`ogrenciNo`	`String` (değişmez)	Doğrudan atandı	`String` değiştirilemez; paylaşım güvenlidir
`ad`	`String` (değişmez)	Doğrudan atandı	Aynı gerekçe
`yas`	`int` (ilkel)	Değer kopyalandı	İlkel türde paylaşım kavramı yoktur
`notlar`	`int[]` (**değişebilir**)	`clone()` ile çoğaltıldı	Paylaşılırsa kopyaya yapılan değişiklik aslını bozar
`notAdedi`	`int` (ilkel)	Değer kopyalandı	Aynı gerekçe

Tablo 5.6 — Kopya yapıcıda her alan için verilen karar

> ✓ **İPUCU**
>
> Kopya yapıcı yazarken izlenecek yol basittir: alanları tek tek gözden geçirin ve her biri için "bu tür **değiştirilebilir** mi?" sorusunu sorun. Yanıt evetse kopyalayın, hayırsa doğrudan atayın. `String`, `Integer`, `LocalDate` ve Hafta 4'te yazdığımız `ParaBirimi` gibi değişmez türler ikinci gruptadır.

4.2. Sığ Kopya ve Derin Kopya

Kopyanın "derinliği", nesne türü alanların da çoğaltılıp çoğaltılmadığını anlatır. Sığ (shallow) kopyada yalnızca referanslar kopyalanır; derin (deep) kopyada işaret edilen nesneler de üretilir.



Şekil 5.4 — Sığ kopya diziyi paylaştırır, derin kopya çoğaltır

Laboratuvar `Personel` sınıfı bu farkı bilerek yan yana koyar. Aşağıdaki yapıcı, `sigKopya` bayrağına göre iki davranıştan birini seçer:

Personel.java

```java
/* --- 3. SIG KOPYA YAPICI (yanlis ornek, bilerek boyle) --- */
public Personel(Personel kaynak, boolean sigKopya) {
    this.sicil = kaynak.sicil;
    this.ad = kaynak.ad;
    this.maas = kaynak.maas;
    this.gorevAdedi = kaynak.gorevAdedi;
    if (sigKopya) {
        this.gorevler = kaynak.gorevler;          // AYNI diziyi paylasir
    } else {
        this.gorevler = kaynak.gorevler.clone();  // YENI dizi
    }
}

/* --- 4. DERIN KOPYA YAPICI (dogru varsayilan) --- */
public Personel(Personel kaynak) {
    this(kaynak, false);
}
```

Kod 5.8 — Aynı sınıfta sığ ve derin kopya yan yana

```
▸ TERMINAL — java Gorev2_PersonelDemo (iki kopyanın karşılaştırması)
=== 2) DERIN kopya: diziler AYRI ===
  Kaynak : Personel[P-1001, Betul Mumcu, 42000.00 TL, gorev={Laboratuvar, Odev degerlendirme}]
  Kopya  : Personel[P-1001, Betul Mumcu (yedek kayit), 46200.00 TL, gorev={Laboratuvar (2.
grup), Odev degerlendirme}]
  Kopyanin 0. gorevi degisti, KAYNAGINKI DEGISMEDI.

=== 3) SIG kopya: dizi PAYLASILIYOR ===
  Kopyalama sonrasi:
    Kaynak : Teori dersi, Danismanlik
    Kopya  : Teori dersi, Danismanlik
  Kopyanin 0. gorevi degistirildi:
    Kaynak : UZAKTAN teori dersi, Danismanlik
    Kopya  : UZAKTAN teori dersi, Danismanlik
  KAYNAK DA DEGISTI -> iki nesne AYNI diziyi gosteriyor.
```

Çıktının 3. bölümü tartışmayı sonlandırır: sığ kopyada kopyanın 0. görevini değiştirmek **kaynağın da** 0. görevini değiştirmiştir. İki nesne farklı görünmekte, ama aynı diziyi göstermektedir. Derin kopyada ise böyle bir sızıntı yoktur.

> **▲ DİKKAT**
>
> `clone()` metodu dizilerde **tek düzey** kopya üretir. `int[]` gibi ilkel dizilerde bu yeterlidir. Ancak `Ogrenci[]` gibi nesne dizilerinde `clone()` yalnızca referans dizisini çoğaltır; içindeki `Ogrenci` nesneleri hâlâ paylaşılır. Böyle durumlarda döngüyle eleman eleman kopya yapıcı çağırmak gerekir.

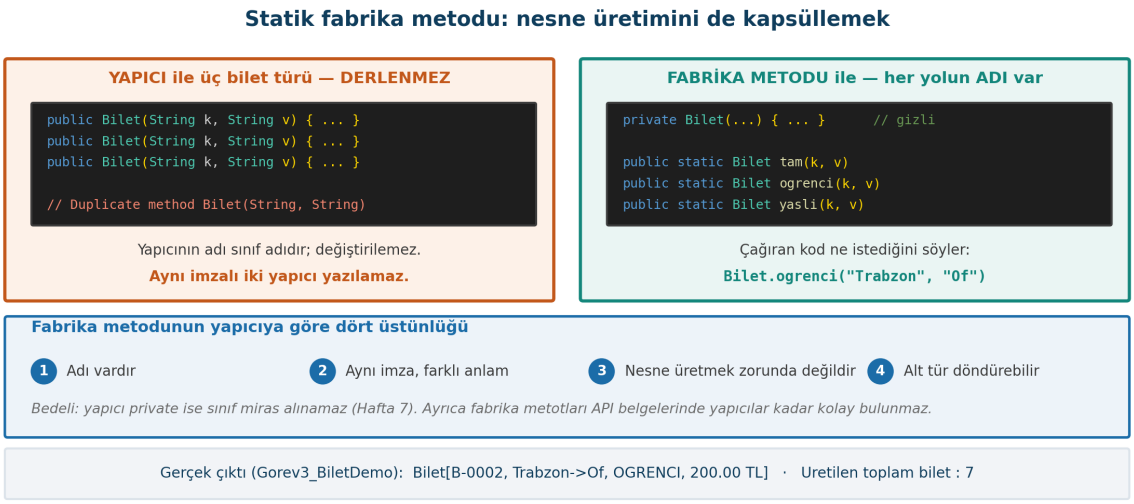
Durum	Sığ kopya yeterli mi?	Gerekçe
Tüm alanlar ilkel (`int`, `double`, `boolean`)	Evet	Değerler zaten kopyalanır

Durum	Sığ kopya yeterli mi?	Gerekçe
Alanlar değişmez nesneler (`String`, `LocalDate`)	Evet	İçerik değiştirilemediği için paylaşım zararsızdır
İlkel dizi (`int[]`, `double[]`)	Hayır — `clone()` yeterli	Dizi değişebilir; tek düzey kopya çözer
Nesne dizisi (`Ogrenci[]`)	Hayır — `clone()` de yetmez	Elemanlar da tek tek kopyalanmalıdır
Değişebilir özel sınıf alanı	Hayır	O sınıfın kopya yapıcısı çağrılmalıdır

Tablo 5.7 — Hangi durumda hangi kopya derinliği gerekir?

4.3. Statik Fabrika Metodu

Bölüm 3.3'te yapıcının doğal bir sınırını görmüştük: aynı imzayla farklı anlamlar üretilemiyordu. Statik fabrika metodu bu sınırı kaldırır ve bunu yaparken nesne üretiminin kendisini de kapsülleme altına alır.



Şekil 5.5 — Statik fabrika metodu: her üretim yolunun bir adı olur

Kayit.java

```java
public class Kayit {

    public static final Kayit BOS = new Kayit("-", "BOS", 0);

    private final String ogrenciNo;
    private final String dersKodu;
    private final int kredi;

    /* Yapici PRIVATE: disaridan new Kayit(...) yazilamaz. */
    private Kayit(String ogrenciNo, String dersKodu, int kredi) {
        this.ogrenciNo = ogrenciNo;
        this.dersKodu = dersKodu;
        this.kredi = kredi;
    }
```

Kod 5.9 — `private` yapıcı: dışarıdan `new Kayit(...)` yazılamaz

3. satırdaki `public static final Kayit BOS` alanı, sınıfın kendi içinde ürettiği ve dışarıya paylaştığı hazır bir nesnedir. 10. satırdaki `private` yapıcı ise kapıyı kapatır: `new Kayit(...)` yazmaya çalışan kod `The constructor Kayit(String, String, int) is not visible` hatasıyla karşılaşır.

Kayit.java

```
1       /* --- Fabrika 1: normal kayit --- */
2       public static Kayit olustur(String ogrenciNo, String dersKodu, int kredi) {
3           if (ogrenciNo == null || ogrenciNo.isBlank()) {
4               System.out.println("  RED: ogrenci numarasi bos olamaz -> BOS dondu");
5               return BOS;
6           }
7           if (kredi < 1 || kredi > 10) {
8               System.out.println("  RED: kredi 1-10 araliginda olmali -> BOS dondu");
9               return BOS;
10          }
11          return new Kayit(ogrenciNo.trim(), dersKodu.toUpperCase(), kredi);
12      }
13
14      /* --- Fabrika 2: ayni imza, farkli anlam (yapici ile MUMKUN DEGIL) --- */
15      public static Kayit dinleyici(String ogrenciNo, String dersKodu) {
16          return new Kayit(ogrenciNo, dersKodu.toUpperCase(), 0);
17      }
```

Kod 5.10 — İki fabrika metodu: `olustur()` ve `dinleyici()`

Bu iki metodun imzaları farklıdır, ama önemli olan bu değildir. Önemli olan, `dinleyici()` metodunun yapıcıyla **yazılamayacak** bir ayrımı adlandırabilmesidir: “kredisi sıfır olan kayıt” bir iş kavramıdır ve adı vardır.

Kayit.java

```
1       /* --- Fabrika 4: kopya (kopya yapicinin fabrika karsiligi) --- */
2       public static Kayit kopyasi(Kayit kaynak) {
3           if (kaynak == BOS) {
4               return BOS;                 // yeni nesne uretmeye gerek yok
5           }
6           return new Kayit(kaynak.ogrenciNo, kaynak.dersKodu, kaynak.kredi);
7       }
```

Kod 5.11 — Fabrika metodu nesne üretmek zorunda değildir

3–4. satırlar, fabrika metodunun yapıcıya göre en az bilinen ama en güçlü üstünlüğünü gösterir: kaynak zaten `BOS` ise **yeni nesne üretilmez**, var olan döndürülür. Bir yapıcı bunu yapamaz; `new` her çağrıda kaçınılmaz olarak bellek ayırır.

```
▸ TERMINAL — java KayitDemo (ret ve nesne paylaşımı)
=== 3) Gecersiz istekte BOS nesnesi doner ===
  RED: ogrenci numarasi bos olamaz -> BOS dondu
  RED: kredi 1-10 araliginda olmali -> BOS dondu
  k4 : Kayit[-, BOS, 0 kredi] -> bosMu=true
  k5 : Kayit[-, BOS, 0 kredi] -> bosMu=true
  Ayni nesne mi? true

=== 4) Fabrika, nesne uretmek ZORUNDA degildir ===
  kopya1 yeni nesne mi? true
```

```
kopya2 == BOS ?       true
Yapici her cagrida MUTLAKA yeni nesne uretirdi.
```

#	Fabrika metodunun üstünlüğü	Somut örnek	Yapıcı yapabilir mi?
1	Her üretim yolunun **adı** vardır	`Bilet.ogrenci(...)`	Hayır
2	Aynı imzayla farklı anlam üretilebilir	`tam()` / `ogrenci()` / `yasli()`	Hayır
3	Nesne üretmek zorunda değildir	`Kayit.kopyasi(BOS)` → `BOS`	Hayır
4	Alt tür döndürebilir	Hafta 9–10'da görülecek	Hayır
5	Girdiyi ayrıştırıp üretebilir	`Kayit.metinden("376203;yzm205;3")`	Kısıtlı

Tablo 5.8 — Statik fabrika metodunun yapıcıya göre üstünlükleri

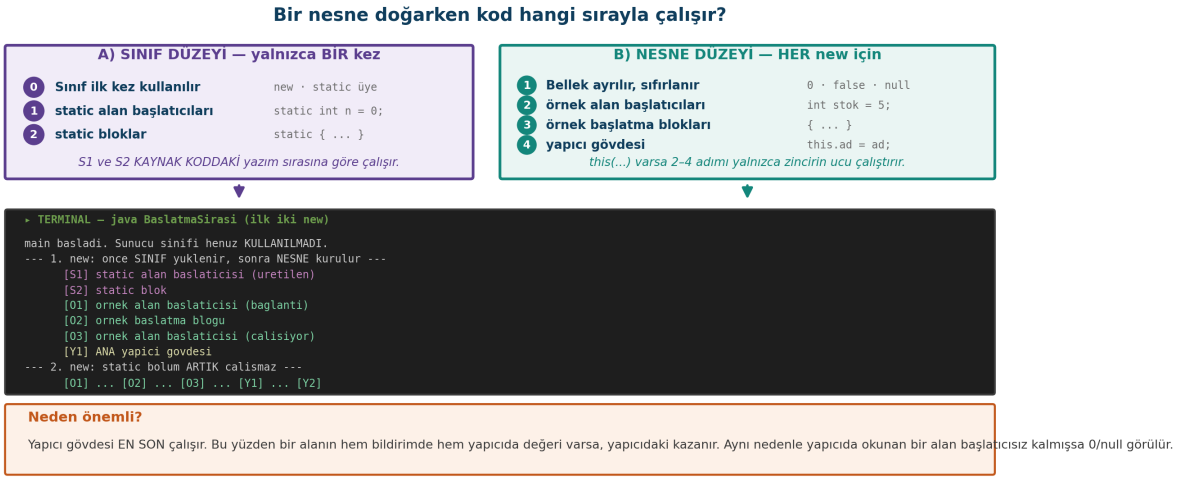
Bedel	Açıklama
Yapıcı `private` ise sınıf miras alınamaz	Alt sınıf üst yapıcıyı çağıramaz (Hafta 7). Çoğu zaman bu bilinçli bir tercihtir.
Fabrika metotları belgelerde kolay bulunmaz	Yapıcılar API belgelerinde ayrı bir bölümde listelenir; fabrikalar sıradan metotlar arasında kalır.
Adlandırma disiplini gerektirir	`of`, `valueOf`, `getInstance`, `newInstance` gibi yerleşik geleneklere uymak okunabilirliği artırır.

Tablo 5.9 — Statik fabrika metodunun bedelleri

> **ⓘ BİLGİ**
>
> Java kitaplığı bu deseni yoğun kullanır: `Integer.valueOf(42)`, `List.of(1, 2, 3)`, `LocalDate.now()`, `String.valueOf(3.14)`. `Integer.valueOf()` üstelik 3. üstünlüğü de kullanır: −128…127 aralığındaki değerler için önbellekteki nesneyi döndürür, yeni nesne üretmez.

4.4. Nesne Başlatma Sırası

Bir sınıfta değer atamanın dört ayrı yeri vardır: `static` alan başlatıcıları, `static` bloklar, örnek alan başlatıcıları ve örnek başlatma blokları — buna yapıcı gövdesini de eklersek beş. Bunların hangi sırayla çalıştığını bilmek, ilk bakışta akademik bir ayrıntı gibi görünür; ancak beklenmedik `0` ve `null` değerlerinin en sık nedeni tam olarak bu sıradır.



Şekil 5.6 — Sınıf düzeyi ve nesne düzeyi başlatmanın sırası

BaslatmaSirasi.java

```java
class Sunucu {

    /* --- A) SINIF DUZEYI: yalnizca bir kez --- */
    static int uretilen = izle("[S1] static alan baslaticisi (uretilen)", 0);

    static {
        izle("[S2] static blok", 0);
    }

    /* --- B) NESNE DUZEYI: her new icin --- */
    int baglanti = izle("[O1] ornek alan baslaticisi (baglanti)", 0);

    {
        izle("[O2] ornek baslatma blogu", 0);
    }

    boolean calisiyor = izleB("[O3] ornek alan baslaticisi (calisiyor)");
```

Kod 5.12 — Gözlenen sınıf: her aşamaya bir iz bırakılmış

BaslatmaSirasi.java

```java
    /* --- Yapici 1: parametresiz --- */
    Sunucu() {
        this("yedek");
        izle("[Y3] parametresiz yapici govdesi (this'ten SONRA)", 0);
    }

    /* --- Yapici 2: tek parametreli --- */
    Sunucu(String ad) {
        this(ad, 80);
        izle("[Y2] tek parametreli yapici govdesi (this'ten SONRA)", 0);
    }

    /* --- Yapici 3: ANA yapici --- */
    Sunucu(String ad, int port) {
        izle("[Y1] ANA yapici govdesi", 0);
        this.ad = ad;
        this.port = port;
```

```
        uretilen++;
    }
```

Kod 5.13 — Üç yapıcı: ikisi `this(...)` ile devrediyor

```
▸ TERMINAL — java BaslatmaSirasi
main basladi. Sunucu sinifi henuz KULLANILMADI.

--- 1. new: once SINIF yuklenir, sonra NESNE kurulur ---
      [S1] static alan baslaticisi (uretilen)
      [S2] static blok
      [O1] ornek alan baslaticisi (baglanti)
      [O2] ornek baslatma blogu
      [O3] ornek alan baslaticisi (calisiyor)
      [Y1] ANA yapici govdesi
  Sonuc: Sunucu[web-01:8080, baglanti=0, calisiyor=true]

--- 2. new: static bolum ARTIK calismaz ---
      [O1] ornek alan baslaticisi (baglanti)
      [O2] ornek baslatma blogu
      [O3] ornek alan baslaticisi (calisiyor)
      [Y1] ANA yapici govdesi
      [Y2] tek parametreli yapici govdesi (this'ten SONRA)
  Sonuc: Sunucu[db-01:80, baglanti=0, calisiyor=true]
```

Çıktıdan çıkarılacak dört sonuç vardır:

1. **`[S1]` ve `[S2]` yalnızca bir kez çalıştı.** İkinci ve üçüncü `new` çağrılarında static bölüm hiç görünmez; sınıf zaten yüklenmiştir.
2. **Sınıf, `main` başladığında değil ilk kullanıldığında yüklendi.** `main basladi` satırı `[S1]`'den öncedir. Sınıf yükleme *tembel* (lazy) yapılır.
3. **`[O1]`, `[O2]`, `[O3]` her `new` için yeniden çalıştı** ve kaynak koddaki **yazım sırasını** izledi: alan, blok, alan.
4. **Yapıcı gövdesi her zaman en sonda çalıştı.** `this(...)` zincirinde bile örnek bölümü yalnızca bir kez ve zincirin ucundaki yapıcıdan önce çalışır.

Sıra	Ne çalışır?	Kaç kez?	Tipik kullanım
0	Sınıf yüklenir (ilk kullanımda)	Sınıf başına 1	—
1	`static` alan başlatıcıları	Sınıf başına 1	Sabitler, sayaçlar
2	`static` bloklar	Sınıf başına 1	Karmaşık sabit kurulumu
3	Bellek ayrılır, alanlar sıfırlanır	Nesne başına 1	`int`→0, `boolean`→`false`, referans→`null`
4	Örnek alan başlatıcıları	Nesne başına 1	Basit varsayılanlar
5	Örnek başlatma blokları	Nesne başına 1	Tüm yapıcılarda ortak olan kurulum
6	Yapıcı gövdesi	Nesne başına 1	Doğrulama ve asıl atama

Tablo 5.10 — Nesne başlatma sırasının tam listesi

✕ SIK YAPILAN HATA

Sıranın en sık yol açtığı hata şudur: bir örnek alan başlatıcısında, henüz atanmamış başka bir alanı okumak. `int b = a * 2;` satırı `a` alanının **altında** yazılmışsa `a` hâlâ 0'dır ve `b` de 0 olur — üstelik derleyici hiçbir uyarı vermez. Kural: örnek alan başlatıcıları birbirine bağımlı olmamalıdır; bağımlılık varsa iş yapıcıya taşınmalıdır.

✓ İPUCU

Örnek başlatma bloğu (`{ ... }`) pratikte az kullanılır; çünkü aynı işi tüm yapıcıların devrettiği bir ana yapıcı zaten yapar. Öğrenilmesinin nedeni, sırayı anlamak ve başkasının kodunda görüldüğünde şaşırmamaktır.

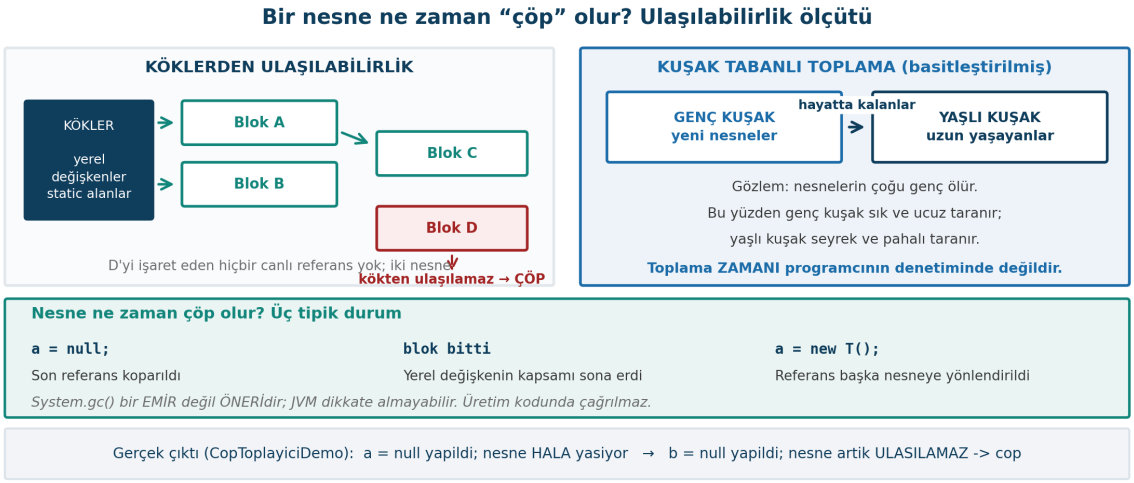
4.5. Yıkıcı Metot: Java'da Neden Yoktur?

C++ bilen bir programcının bu noktada beklediği şey bellidir: yapıcının karşıtı olan, nesne yok edilirken çalışan bir **yıkıcı (destructor)** metot. Java'da böyle bir metot **yoktur** ve bu bir eksiklik değil, bilinçli bir tasarım kararıdır.



Şekil 5.7 — Belleği C'de programcı, Java'da çöp toplayıcı geri alır

Nedeni basittir: yıkıcının asıl işi belleği geri vermektir; Java'da ise belleği **çöp toplayıcı (garbage collector)** geri alır. Bir nesnenin ne zaman yok edileceği programcının denetiminde olmadığı için, o anda çalışacak bir metodun sözü de verilemez.



Şekil 5.8 — Bir nesne kökten ulaşılamaz hâle geldiğinde çöp olur

CopToplayiciDemo.java

```
1       public static void main(String[] args) {
2           System.out.println("=== 1) Referans sayisi ve ulasilabilirlik ===");
3           Blok a = new Blok("A", 64);
4           Blok b = a;                      // ayni nesneye ikinci referans
5           System.out.println("  a == b ? " + (a == b) + "  -> tek nesne, iki referans");
6           a = null;
7           System.out.println("  a = null yapildi; nesne HALA yasiyor: " + b);
8           b = null;
9           System.out.println("  b = null yapildi; nesne artik ULASILAMAZ -> cop");
10
11          System.out.println();
12          System.out.println("=== 2) Kapsam disina cikan nesne otomatik cop olur ===");
13          {
14              Blok gecici = new Blok("Gecici", 32);
15              System.out.println("  Blok icinde  : " + gecici);
16          }
17          System.out.println("  Blok bitti; 'gecici' adi artik yok -> nesne cop");
```

Kod 5.14 — Ulaşılabilirlik: referans koparmak ve kapsam dışına çıkmak

```
▸ TERMINAL — java CopToplayiciDemo
=== 1) Referans sayisi ve ulasilabilirlik ===
  URETILDI : A (64 KB)
  a == b ? true  -> tek nesne, iki referans
  a = null yapildi; nesne HALA yasiyor: Blok[A, 64 KB]
  b = null yapildi; nesne artik ULASILAMAZ -> cop

=== 2) Kapsam disina cikan nesne otomatik cop olur ===
  URETILDI : Gecici (32 KB)
  Blok icinde  : Blok[Gecici, 32 KB]
  Blok bitti; 'gecici' adi artik yok -> nesne cop

=== 3) Cop toplama ZAMANI programcinin elinde degildir ===
  200 gecici nesne uretildi, hicbiri tutulmadi.
  System.gc() cagrildi - bu bir EMIR degil, ONERIDIR.
  JVM cop toplamayi hic yapmayabilir de.
```

Konu	C / C++	Java
Bellek geri verme	`free()` / `delete` — programcı	Çöp toplayıcı — JVM
Zamanlama	Kesin ve öngörülebilir	Belirsiz
Yıkıcı metot	`~Sinif()` — vardır ve garantilidir	**Yoktur**
Bellek sızıntısı	`free()` unutulursa oluşur	Ulaşılabilirlik varsa oluşur
Çift serbest bırakma	Çökmeye yol açar	Kavram olarak yoktur
Sarkan işaretçi	Tanımsız davranış	Kavram olarak yoktur
Dış kaynak (dosya, soket)	Yıkıcıda kapatılır (RAII)	`close()` + `try-with-resources`
`finalize()`	—	Java 9'dan beri kullanımdan kaldırıldı

Tablo 5.11 — Bellek ve kaynak yönetiminin karşılaştırması

> **▲ DİKKAT**
>
> `finalize()` metodu bir yıkıcı **değildir** ve yıkıcı yerine kullanılamaz. Üç kusuru vardır: (i) ne zaman çağrılacağı belirsizdir, (ii) program sonlanırsa hiç çağrılmayabilir, (iii) çağrıldığında bile nesneyi yeniden canlandırabilir ve çöp toplamayı geciktirir. Bu nedenle Java 9'dan beri *deprecated* işaretlidir ve yeni kodda **kullanılmaz**.

> **ⓘ BİLGİ**
>
> Çöp toplayıcı yalnızca **belleği** geri alır. Dosya tanıtıcısı, ağ soketi, veritabanı oturumu, ekran kilidi gibi kaynaklar işletim sisteminden alınır ve JVM'in muhasebesinde görünmez. Bunlar için hâlâ açık bir "kapatma" eylemi gerekir — bir sonraki bölümün konusu budur.

4.6. `AutoCloseable` ve `try-with-resources`

Java'nın yıkıcı yerine sunduğu çözüm iki parçadan oluşur: kaynağı tutan sınıf `AutoCloseable` arayüzünü gerçekler ve `close()` metodunu yazar; kullanan kod ise nesneyi `try-with-resources` bloğunda üretir. Derleyici, blok nasıl biterse bitsin `close()` çağrısını ekler.

● *DosyaKaynagi.java*

```java
public class DosyaKaynagi implements AutoCloseable {

    private final String ad;
    private boolean acik;

    public DosyaKaynagi(String ad) {
        this.ad = ad;
        this.acik = true;
        System.out.println("    ACILDI  : " + ad);
    }
```

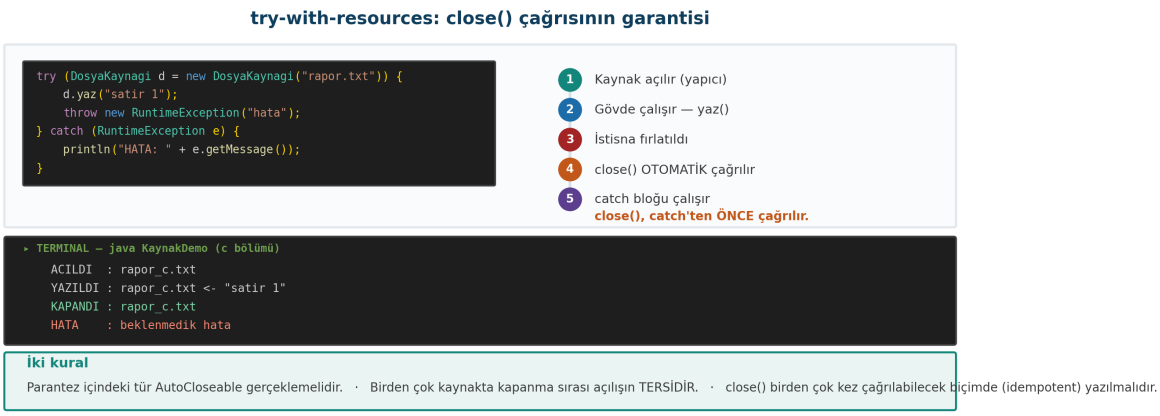
Kod 5.15 — `AutoCloseable` gerçekleyen sınıf — yapıcı kaynağı açar

● *DosyaKaynagi.java*

```
1       /* close() metodu AutoCloseable'dan gelir; try-with-resources onu
2          blok bittiginde OTOMATIK ve GARANTILI olarak cagirir. */
3       @Override
4       public void close() {
5           if (acik) {
6               acik = false;
7               System.out.println("    KAPANDI : " + ad);
8           }
9       }
```

Kod 5.16 — `close()` metodu — birden çok kez çağrılabilir (idempotent)

`close()` metodunun 5. satırdaki `if (acik)` denetimi önemsiz görünür ama zorunludur: bir kaynak yanlışlıkla iki kez kapatılırsa ikinci çağrının **etkisiz** olması beklenir. Buna *idempotent* davranış denir.



Şekil 5.9 — `try-with-resources` yaşam çizgisi: `close()` `catch`'ten önce çalışır

Aynı işi üç farklı biçimde yazıp karşılaştıralım. `KaynakDemo` sınıfı üç yöntemi de aynı senaryoda (gövde içinde istisna fırlatılıyor) çalıştırır.

KaynakDemo.java

```
1       private static void elleKapat() {
2           DosyaKaynagi d = new DosyaKaynagi("rapor_a.txt");
3           try {
4               d.yaz("satir 1");
5               if (d.acikMi()) {
6                   throw new RuntimeException("beklenmedik hata");
7               }
8               d.close();                      // buraya HIC ULASILAMAZ
9           } catch (RuntimeException e) {
10              System.out.println("    HATA    : " + e.getMessage());
11          }
12          System.out.println("    Durum   : " + d);
13          System.out.println("    Sonuc   : dosya ACIK kaldi -> KAYNAK SIZINTISI");
14      }
```

Kod 5.17 — (a) Elle kapatma — istisna atlarsa `close()` satırına ulaşılmaz

KaynakDemo.java

```
1       private static void tryWithResources() {
2           DosyaKaynagi disKopya = null;
```

```
3           try (DosyaKaynagi d = new DosyaKaynagi("rapor_c.txt")) {
4               disKopya = d;
5               d.yaz("satir 1");
6               throw new RuntimeException("beklenmedik hata");
7           } catch (RuntimeException e) {
8               System.out.println("    HATA    : " + e.getMessage());
9           }
10          System.out.println("    Durum   : " + disKopya);
11          System.out.println("    Sonuc   : close() catch'ten ONCE otomatik cagrildi");
12      }
```

Kod 5.18 — (c) `try-with-resources` — `close()` derleyici tarafından eklenir

```
▸ TERMINAL — java KaynakDemo (üç yöntem)
=== (a) ELLE KAPATMA - hata olursa sizinti ===
    ACILDI  : rapor_a.txt
    YAZILDI : rapor_a.txt <- "satir 1"
    HATA    : beklenmedik hata
    Durum   : DosyaKaynagi[rapor_a.txt, acik=true]
    Sonuc   : dosya ACIK kaldi -> KAYNAK SIZINTISI

=== (b) TRY-FINALLY - kapanir, kod uzun ===
    ACILDI  : rapor_b.txt
    YAZILDI : rapor_b.txt <- "satir 1"
    HATA    : beklenmedik hata
    KAPANDI : rapor_b.txt
    Durum   : DosyaKaynagi[rapor_b.txt, acik=false]
    Sonuc   : kapandi, ama 5 satir fazladan kod yazildi

=== (c) TRY-WITH-RESOURCES - kisa ve garantili ===
    ACILDI  : rapor_c.txt
    YAZILDI : rapor_c.txt <- "satir 1"
    KAPANDI : rapor_c.txt
    HATA    : beklenmedik hata
    Durum   : DosyaKaynagi[rapor_c.txt, acik=false]
    Sonuc   : close() catch'ten ONCE otomatik cagrildi
```



Şekil 5.10 — Aynı iş, üç farklı kapatma yöntemi ve sonuçları

Çıktıda dikkat edilmesi gereken iki nokta vardır. Birincisi, (a) yönteminde `acik=true` kalmıştır: kaynak sızmıştır. İkincisi, (c) yönteminde `KAPANDI` satırı `HATA` satırından **önce** yazılmıştır; yani `close()` çağrısı `catch` bloğundan önce yapılmıştır.

```
▸ TERMINAL — java KaynakDemo (birden çok kaynak)
=== (d) BIRDEN COK KAYNAK: kapanma sirasi TERSTIR ===
    ACILDI  : giris.txt
    ACILDI  : cikis.txt
    YAZILDI : giris.txt <- "okundu"
    YAZILDI : cikis.txt <- "yazildi"
    KAPANDI : cikis.txt
    KAPANDI : giris.txt
    Sonuc   : acilis giris->cikis, kapanis cikis->giris
```

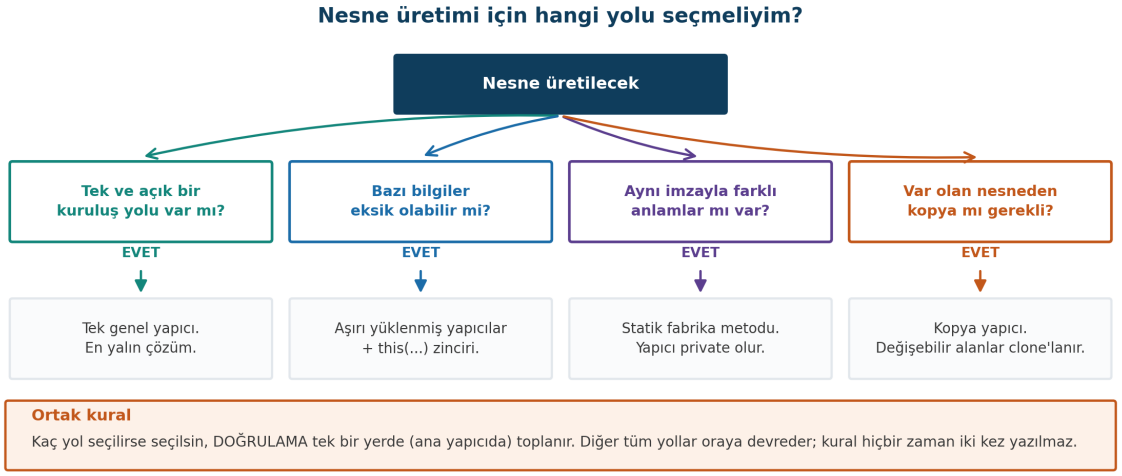
Son bölüm, birden çok kaynak kullanıldığında kapanma sırasının **açılışın tersi** olduğunu gösterir. Bu, kaynaklar arasında bağımlılık olduğunda (örneğin bir akışın başka bir akışı sarması durumunda) doğru davranıştır.

Kural	Açıklama
Tür `AutoCloseable` gerçeklemelidir	Aksi hâlde `The resource type X does not implement AutoCloseable` hatası alınır
Değişken bildirimi parantez içinde olmalıdır	Java 9'dan itibaren, zaten `final` olan dış değişken de yazılabilir
`close()` elle çağrılmaz	Blok zaten çağırır; elle çağırmak iki kez kapatma denemesi olur
`close()` idempotent yazılmalıdır	İkinci çağrı hata üretmemeli, sessizce dönmelidir
Kapanma sırası açılışın tersidir	Sonra açılan önce kapanır — bağımlılık zinciri korunur
`close()` istisna fırlatabilir	Gövdedeki istisna asıl, `close()`'unki *bastırılmış* (suppressed) sayılır

Tablo 5.12 — `try-with-resources` kullanım kuralları

4.7. Hangi Üretim Yolunu Seçmeli?

Bu haftada dört ayrı nesne üretim aracı gördük. Aralarında seçim yapmak için izlenecek akış aşağıdadır.



Şekil 5.11 — Nesne üretimi için karar akışı

Akışın dört kolu birbirini dışlamaz; bir sınıf aynı anda hem aşırı yüklenmiş yapıcılara hem kopya yapıcıya sahip olabilir. Değişmeyen tek şey, hepsinin **aynı ana yapıcıya** devretmesidir.

5. C'den Java'ya Köprü

⇄ C DİLİNDEN JAVA'YA KÖPRÜ

C dilinde bir `struct` için "kurucu" yazmak bir gelenektir, dilin kuralı değildir: `hesap_init(&h, ...)` biçiminde bir fonksiyon yazılır ve **çağrılması umulur**. Java'da yapıcı dilin kendisi tarafından zorunlu kılınır: `new` ifadesi yapıcıyı çağırmadan nesne üretemez. Aradaki fark, "başlatılmamış nesne" diye bir durumun Java'da bulunmamasıdır.

ogrenci.c

```c
/* C - baslatma bir GELENEKTIR, zorunluluk degildir */
typedef struct { char no[16]; char ad[64]; int yas; } Ogrenci;

int ogrenci_init(Ogrenci* o, const char* no, const char* ad, int yas) {
    if (no == NULL || ad == NULL) return 0;
    if (yas < 17 || yas > 70)     return 0;
    strncpy(o->no, no, 15); strncpy(o->ad, ad, 63); o->yas = yas;
    return 1;
}

int main(void) {
    Ogrenci o;                    /* alanlar COP DEGER icerir */
    printf("%d\n", o.yas);        /* tanimsiz davranis - derleyici izin verir */
    ogrenci_init(&o, "376201", "Elif Kaya", 20);   /* cagrilmasi UMULUR */
    return 0;
}
```

Kod 5.19 — C'de başlatma: çağrılması umulan bir fonksiyon

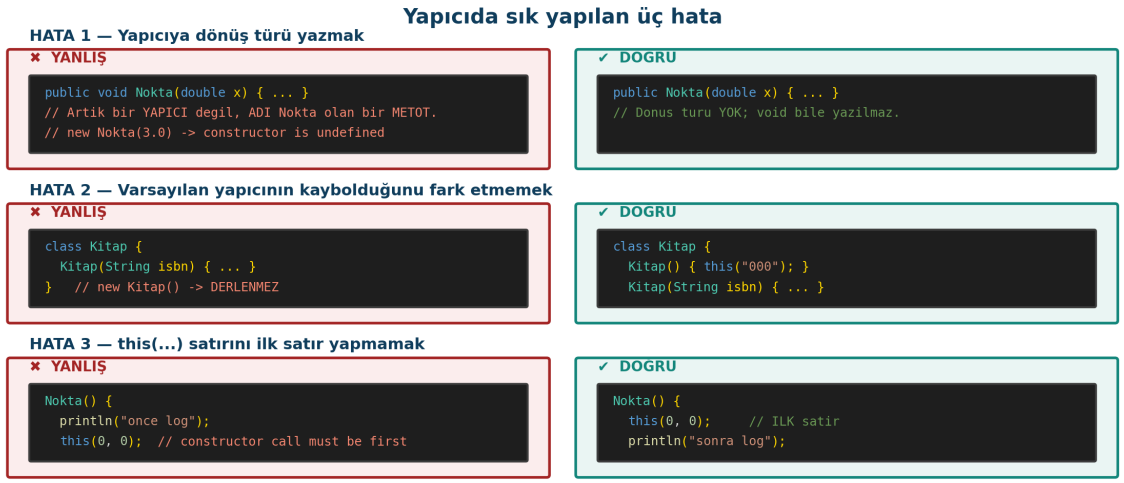
Konu	C dilinde	Java'da
Başlatma zorunluluğu	Yok — `init` çağrılmayabilir	Var — `new` yapıcıyı çağırır
Başlatılmamış nesne	Mümkün (çöp değerler)	**İmkânsız**
Birden çok kuruluş yolu	`init_a`, `init_b` gibi ayrı adlar	Aşırı yüklenmiş yapıcılar
Ortak kodun paylaşımı	Bir fonksiyon diğerini çağırır	`this(...)` zinciri
Kopyalama	`memcpy` ya da atama — bayt kopyası	Kopya yapıcı (atama yalnızca referans kopyalar)
Bellek ayırma	`malloc` — programcı	`new` — JVM
Bellek geri verme	`free` — programcı	Çöp toplayıcı
Yıkım anı	`free` çağrısı — kesin	Belirsiz
Dosya/soket kapatma	`fclose` elle, `goto cleanup` deseni	`try-with-resources` — derleyici ekler
Hata bildirimi	Dönüş kodu (`0` / `-1`)	İstisna (Hafta 11)

Tablo 5.13 — C'deki başlatma/yıkım ile Java'nın karşılaştırması

Tablonun `memcpy` satırı özellikle önemlidir. C'de `Ogrenci b = a;` satırı tüm baytları kopyalar; iki bağımsız yapı oluşur. Java'da aynı görünen `Ogrenci b = a;` satırı yalnızca **referansı** kopyalar; ortada tek nesne kalır. C'den gelen öğrencilerin en sık yaptığı hata budur ve derleyici hiçbir uyarı vermez.

> **ⓘ BİLGİ**
>
> C++ bilenler için not: C++'ın **RAII** (Resource Acquisition Is Initialization) deseni, kaynağı yapıcıda alıp yıkıcıda bırakmaya dayanır ve yıkıcı kapsam sonunda **garantili** çalışır. Java'nın `try-with-resources` yapısı, aynı güvenceyi yıkıcı olmadan sağlamak için tasarlanmıştır: garanti nesnenin yaşam süresine değil, **bloğun** kapsamına bağlanır.

6. Sık Yapılan Hatalar



Şekil 5.12 — Yapıcıda en sık yapılan üç hata ve düzeltilmiş hâlleri

6.1. Yapıcıya Dönüş Türü Yazmak

> **✖ HATA 1 — Sessizce metoda dönüşen yapıcı**
>
> `public void Nokta(double x)` yazıldığında ortaya bir yapıcı değil, **adı Nokta olan sıradan bir metot** çıkar. Derleyici bundan şikâyet etmez; hata ancak `new Nokta(3.0)` çağrısında `The constructor Nokta(double) is undefined` iletisiyle görünür. Yapıcının dönüş türü yoktur; `void` bile yazılmaz.

6.2. Varsayılan Yapıcının Kaybolduğunu Fark Etmemek

> **✖ HATA 2 — Kaybolan varsayılan yapıcı**
>
> Sınıfa ilk yapıcıyı yazdığınız anda, derleyicinin eklediği parametresiz yapıcı **kaybolur**. `Kitap(String isbn)` yazdıktan sonra `new Kitap()` derlenmez. Parametresiz kuruluş hâlâ isteniyorsa açıkça yazılmalı ve tercihen `this(...)` ile ana yapıcıya devretmelidir.

6.3. `this(...)` Satırını İlk Satır Yapmamak

✕ HATA 3 — `this(...)` ilk satır değil

`this(...)` çağrısı yapıcının **ilk** satırı olmak zorundadır. Önüne bir günlük satırı ya da bir atama koymak `Constructor call must be the first statement in a constructor` hatası verir. Nedeni `final` alanlardır: zincir çalışmadan önce alanlara dokunmak, `final` alanın iki kez atanması anlamına gelirdi.

6.4. Kopya Yapıcı Yazıp Derinleştirmemek

✕ HATA 4 — Adı kopya, kendisi paylaşım

Kopya yapıcı yazmak tek başına yetmez. `this.notlar = kaynak.notlar;` satırı bir kopya yapıcının içinde bile olsa **sığ kopya** üretir; iki nesne aynı diziyi paylaşır. Değişebilir her alan için ayrıca kopya üretilmelidir.

```
// YANLIS - kopya yapici gorunumu, paylasim gercegi
public Personel(Personel kaynak) {
    this.sicil = kaynak.sicil;
    this.gorevler = kaynak.gorevler;          // AYNI dizi
}

// DOGRU - degisebilir alan cogaltilir
public Personel(Personel kaynak) {
    this.sicil = kaynak.sicil;
    this.gorevler = kaynak.gorevler.clone();  // YENI dizi
}
```

Kod 5.20 — Sığ ve derin kopya yapıcı yan yana

6.5. Yapıcıda Aşırı İş Yapmak

✕ HATA 5 — Yapıcıyı ağır işlerle doldurmak

Yapıcının işi nesneyi geçerli bir başlangıç durumuna getirmektir; dosya okumak, ağ bağlantısı kurmak ya da uzun hesap yapmak değildir. Yapıcı içinde istisna oluşursa ortada **yarım kurulmuş** bir nesne kalır ve buna erişilemez. Ağır işler ayrı bir metoda (`baslat()`, `yukle()`) ya da bir fabrika metoduna taşınmalıdır.

6.6. Kaynağı `try-with-resources` Dışında Açmak

✕ HATA 6 — Parantezin dışında açılan kaynak

`DosyaKaynagi d = new DosyaKaynagi("a.txt");` satırı `try (...)` parantezinin **dışında** yazıldığında derleyici `close()` eklemez. Bu, Bölüm 4.6'daki (a) yöntemidir ve kaynak sızıntısı üretir. Kaynak her zaman parantezin içinde üretilmelidir.

6.7. Derleyici İletileri Sözlüğü

Derleyici iletisi	Ne demek?	Ne yapmalı?
`The constructor X() is undefined`	Yazdığınız ilk yapıcı varsayılanı sildi	Parametresiz yapıcıyı açıkça yazın
`Constructor call must be the first statement`	`this(...)` ilk satır değil	`this(...)` satırını en başa taşıyın
`Recursive constructor invocation`	Yapıcı zinciri kendine dönüyor	Zincirin ucunda `this(...)` içermeyen bir yapıcı bulunmalı
`Duplicate method X(String, String)`	İki yapıcının imzası aynı	Statik fabrika metodu kullanın
`The constructor X(...) is not visible`	Yapıcı `private`	Sınıfın fabrika metodunu çağırın
`The blank final field ... may not have been initialized`	`final` alan bazı yapıcı yollarında atanmadan kalıyor	Tüm yolların ana yapıcıya devretmesini sağlayın
`The final field ... cannot be assigned`	`final` alana ikinci atama (çoğu zaman `this(...)` sonrası)	Atamayı yalnızca ana yapıcıda yapın
`The resource type X does not implement AutoCloseable`	`try (...)` içine uygunsuz tür konmuş	Sınıfa `implements AutoCloseable` ekleyin

Tablo 5.14 — Bu haftaya özgü derleme hataları ve çözümleri

7. İyi Tasarım İlkeleri

Bu bölümdeki maddeler, dönem boyunca yazacağınız her sınıfta uygulamanız beklenen kod yazım standartlarıdır.

1. **Tek bir ana yapıcı belirleyin.** Tüm parametreleri alan, tüm doğrulamayı barındıran bir yapıcı yazın; diğer yapıcıların hepsi doğrudan ya da dolaylı olarak ona devretsin.
2. **Doğrulamayı asla iki kez yazmayın.** Bir kural iki yapıcıda görünüyorsa zincir kurulmamış demektir.
3. **Yapıcıları kısadan uzuna sıralayın.** Okuyucu, hangi bilginin zorunlu hangisinin isteğe bağlı olduğunu bu sıralamadan çıkarabilmeli.
4. **Yapıcı gövdesini kısa tutun.** Dosya okuma, ağ erişimi ve uzun hesaplar yapıcının işi değildir.
5. **Dışarıdan gelen değişebilir nesneleri yapıcıda kopyalayın.** `this.notlar = notlar.clone();` — aksi hâlde çağıran kod nesnenin içine erişimini sürdürür.
6. **Kopya yapıcı yazıyorsanız her değişebilir alanı çoğaltın.** Aksi hâlde kopya yapıcı adını taşıyan bir paylaşım kodu yazmış olursunuz.
7. **Üç ya da daha fazla üretim yolu varsa statik fabrika düşünün.** Özellikle yollar aynı imzayı gerektiriyorsa fabrika tek seçenektir.
8. **Fabrika metotlarında yerleşik adları kullanın:** `of`, `valueOf`, `from`, `getInstance`, `newInstance`, `copyOf`.
9. **Dış kaynak tutan her sınıf `AutoCloseable` gerçeklesin.** `close()` idempotent olsun; kullanan kod `try-with-resources` kullansın.
10. **`finalize()` yazmayın.** Kullanımdan kaldırılmıştır ve hiçbir güvence sağlamaz.
11. **Örnek alan başlatıcılarını birbirine bağımlı yapmayın.** Bağımlılık varsa iş yapıcıya taşınmalıdır.

7.1. Adlandırma ve Yazım Kuralları

Öğe	Kural	Örnek
Yapıcı	Sınıf adıyla birebir aynı	`public Kitap(...)`
Ana yapıcı	En çok parametreli olan; sınıfın en altına değil, diğerlerinin **hemen ardına** yazılır	`Kitap(isbn, baslik, yazar, sayfa, yil)`
Kopya yapıcı	Tek parametre, sınıfın kendi türü	`Kitap(Kitap kaynak)`
Fabrika — üretim	`of`, `from`, `valueOf`	`Kayit.metinden(...)`
Fabrika — tekil	`getInstance`	`Ayarlar.getInstance()` (Hafta 14)
Fabrika — kopya	`copyOf`, `kopyasi`	`Kayit.kopyasi(k)`
Kaynak sınıfı	`close()` metodu; `AutoCloseable`	`DosyaKaynagi`
Sabit nesne	`public static final` + SCREAMING_SNAKE_CASE	`Kayit.BOS`, `Koordinat.BASLANGIC`

Tablo 5.15 — Yapıcı ve fabrika bağlamında Java adlandırma gelenekleri

8. Bir Sonraki Haftaya Köprü

Bu hafta iki yerde `static` anahtar sözcüğünü kullanmak zorunda kaldık ama açıklamasını erteledik: statik fabrika metotlarında (`public static Bilet tam(...)`) ve `Bilet` sınıfının otomatik numara üreten sayacında (`private static int sonrakiSira`). Hafta 6 tam olarak bu boşluğu dolduracaktır.

Bu haftaki kavram	Hafta 6'da nasıl genişleyecek?
`static` fabrika metodu	`static` metodun ne olduğu; neden nesne gerektirmediği
`private static int sonrakiSira`	Sınıf düzeyi alan: tek kopya, tüm nesnelerce paylaşılan durum
`static` başlatma bloğu	Sınıf yükleme zamanlaması ve statik kurulumun tam anlatımı
`public static final Kayit BOS`	Sabit nesne deseni ve `Math`, `Integer.MAX_VALUE` benzeri yardımcı sınıflar
Nesne sayacı (`uretilen++`)	Sayaç deseni ve statik/örnek ayrımının bellekteki karşılığı

Tablo 5.16 — Hafta 5 kavramlarının Hafta 6'daki karşılıkları

Daha uzun vadede: yapıcı zinciri Hafta 7'de `super(...)` çağrısıyla sınıf hiyerarşisine yayılacak; statik fabrikanın “alt tür döndürme” üstünlüğü Hafta 9–10'da çok biçimlilik ve arayüzlerle anlam kazanacak; `try-with-resources` Hafta 11'de istisna yönetiminin tamamıyla ve Hafta 12'de dosya işlemleriyle birlikte yeniden karşımıza çıkacaktır. Hafta 14'te göreceğimiz **Singleton** ve **Factory** desenleri de doğrudan bu haftanın `private` yapıcı fikrine dayanır.

9. Haftanın Özeti

- **Nesneyi `new` üretir, yapıcı başlatır.** Yapıcının dönüş türü yoktur; çünkü döndüreceği bir şey yoktur.
- **Yapıcılar aşırı yüklenebilir**, ayrım yalnızca imzaya dayanır. Parametre adları imzanın parçası değildir.
- **`this(...)` zinciri doğrulamayı tek noktada toplar.** Çağrı yukarı çıkar, gövdeler aşağı iner. `this(...)` ilk satır olmak zorundadır.
- **Yapıcıda geçersiz değer nesneyi hiç doğurmamalıdır.** Setter'daki “eski geçerli durumu koru” davranışının yapıcıdaki karşılığı budur.
- **Atama nesne kopyalamaz;** kopya yapıcı kopyalar. Değişebilir alanlar için ayrıca `clone()` ya da eleman eleman kopya gerekir (derin kopya).
- **Statik fabrika metodu** üretim yollarını adlandırır, aynı imzayla farklı anlam üretmeyi ve gerektiğinde hiç nesne üretmemeyi mümkün kılar; bedeli mirasa kapalılıktır.
- **Başlatma sırası:** static alan → static blok (sınıf başına bir kez) → bellek sıfırlama → örnek alan → örnek blok → yapıcı gövdesi (her `new` için).
- **Java'da yıkıcı yoktur.** Belleği çöp toplayıcı geri alır; toplama zamanı belirsizdir. `finalize()` kullanımdan kaldırılmıştır.
- **Bellek ≠ kaynak.** Dosya, soket ve benzeri dış kaynaklar için `AutoCloseable` + `try-with-resources` kullanılır; `close()` idempotent olmalıdır.

→ BİR SONRAKİ HAFTA

Hafta 6 — Statik ve Nesne Üyesi Değişkenler. `static` alan ve metotların ne anlama geldiği, sınıf düzeyi ile nesne düzeyinin bellekteki farkı, `static final` sabitler, sayaç deseni ve yardımcı (utility) sınıflar.

10. Alıştırmalar

Alıştırmalar üç kümeye ayrılmıştır: kavram soruları (1–4), kod okuma ve hata bulma (5–7), programlama (8–12). Zorluk göstergesi: ●○○ temel, ●●○ orta, ●●● ileri. Derse gelmeden önce `Nokta`, `Kayit` ve `DosyaKaynagi` sınıflarını gözden geçirmeniz önerilir.

10.1. Kavram Soruları

Alıştırma 1 ●○○

"Nesneyi yapıcı üretmez" cümlesini kendi cümlelerinizle açıklayınız. Bu bilgi, yapıcının dönüş türünün neden bulunmadığını nasıl açıklar? Bölüm 3.1'deki beş adımlı şemayı dayanak olarak kullanınız.

Alıştırma 2 ●●○

`this(...)` çağrısının yapıcının ilk satırı olma zorunluluğunun gerekçesini `final` alanlar üzerinden açıklayınız. Bu kural olmasaydı hangi tutarsızlık ortaya çıkabilirdi?

Alıştırma 3 ●●○

Statik fabrika metodunun yapıcıya göre dört üstünlüğünü ve en az bir bedelini yazınız. Her üstünlük için ders notundan **farklı** birer örnek üretiniz.

Alıştırma 4 ●●●

Java'da yıkıcı bulunmamasının gerekçesini açıklayınız. "Çöp toplayıcı belleği geri alıyorsa `close()` metoduna neden hâlâ ihtiyaç var?" sorusunu somut bir kaynak türü örneğiyle yanıtlayınız.

10.2. Kod Okuma ve Hata Bulma

Alıştırma 5 ●●○

Aşağıdaki sınıfta **üç ayrı** kusur vardır. Üçünü de bulup nasıl düzelteceğinizi yazınız.

```
1  public class Urun {
2      private final String barkod;
```

```
    private String ad;
    private double fiyat;

    public void Urun(String barkod) {              // (1)
        this.barkod = barkod;
    }

    public Urun(String barkod, String ad) {
        this.ad = ad;
        this(barkod);                              // (2)
    }

    public Urun(String barkod, String ad, double fiyat) {
        if (fiyat < 0) throw new IllegalArgumentException("...");
        this.barkod = barkod; this.ad = ad; this.fiyat = fiyat;   // (3)
    }
}
```

Kod 5.21 — Alıştırma 5 için inceleme kodu

Alıştırma 6 ●●●

Aşağıdaki program çalıştırıldığında ekrana ne yazılır? Yanıtınızı **çalıştırmadan önce** yazınız, sonra doğrulayınız. Sıra beklediğinizden farklı çıktıysa nedenini Bölüm 4.4'e dayanarak açıklayınız.

```
public class Sira {
    static int s = yaz("A");
    int x = yaz("B");
    static { yaz("C"); }
    { yaz("D"); }
    int y = yaz("E");

    Sira() { this(1); yaz("F"); }
    Sira(int n) { yaz("G"); }

    static int yaz(String t) { System.out.print(t + " "); return 0; }

    public static void main(String[] args) {
        System.out.print("BASLA ");
        new Sira();
        System.out.print("| ");
        new Sira(9);
    }
}
```

Kod 5.22 — Alıştırma 6 için inceleme kodu

Alıştırma 7 ●●○

Aşağıdaki kopya yapıcı derlenir ve çalışır; ancak amaçladığı işi yapmaz. Sorunu bulunuz, sorunu **gösteren** üç satırlık bir sürücü program yazınız ve düzeltiniz.

```
public class Takim {
```

```
    private final String ad;
    private int[] puanlar;

    public Takim(String ad, int[] puanlar) {
        this.ad = ad;
        this.puanlar = puanlar;
    }

    public Takim(Takim kaynak) {
        this(kaynak.ad, kaynak.puanlar);
    }

    public void puanGuncelle(int i, int p) { puanlar[i] = p; }
    public int puan(int i) { return puanlar[i]; }
}
```

Kod 5.23 — Alıştırma 7 için inceleme kodu

10.3. Programlama Alıştırmaları

Alıştırma 8 ●○○

`Sicaklik` adında bir sınıf yazınız. Üç yapıcısı olsun: `Sicaklik()` (0 °C), `Sicaklik(double celsius)` ve `Sicaklik(double deger, char birim)` — birim `'C'`, `'F'` ya da `'K'` olabilir. Üçü de tek bir ana yapıcıya devretmelidir. Mutlak sıfırın altındaki değerler `IllegalArgumentException` ile reddedilmelidir.

İpucu: Ana yapıcı `Sicaklik(double celsius)` olabilir; üçüncü yapıcı önce çevirme yapıp `this(...)` çağırsın.

Alıştırma 9 ●●●

`Matris` adında, `private final double[][] veri` alanı tutan bir sınıf yazınız. Bir yapıcı dışarıdan `double[][]` alsın, bir kopya yapıcı da olsun. **İkisi de derin kopya yapmalıdır.** İki boyutlu dizide `clone()` çağrısının neden yetmediğini kodunuzun yorum satırında açıklayınız.

İpucu: `veri.clone()` yalnızca satır referanslarını kopyalar; her satır için ayrıca `clone()` gerekir. Bölüm 4.2'deki “nesne dizisi” satırını yeniden okuyunuz.

Alıştırma 10 ●●○

`Renk` adında, `private final int r, g, b` alanları olan bir sınıf yazınız. Yapıcı `private` olsun; sınıf yalnızca şu fabrika metotlarını sunsun: `rgb(int, int, int)`, `gri(int)`, `onaltilik(String)` (`"#FF8800"` biçimi) ve `public static final Renk SIYAH, BEYAZ, KIRMIZI` sabitleri. `gri(0)` çağrısı **yeni nesne üretmeden** `SIYAH` sabitini döndürmelidir.

İpucu: `Kayit` sınıfındaki `BOS` düzenini ve `kopyasi()` metodundaki “nesne üretmeme” hilesini örnek alınız.

Alıştırma 11 ●●○

`Sayac` adında bir sınıf yazınız ve nesne başlatma sırasını **deneysel olarak** doğrulayınız: bir `static` alan başlatıcısı, bir `static` blok, iki örnek alan başlatıcısı, bir örnek blok ve `this(...)` ile zincirlenmiş üç yapıcı yazın.

Her aşamada bir satır yazdırın. Ardından çıktıyı Bölüm 4.4'teki tabloyla karşılaştıran kısa bir rapor yazınız.

İpucu: `BaslatmaSirasi.java` dosyasındaki `Sunucu` sınıfını örnek alabilirsiniz; ancak aşama adlarını kendiniz belirleyiniz.

Alıştırma 12 ●●●

`Baglanti` adında, `AutoCloseable` gerçekleyen bir sınıf yazınız. Sınıf açık bağlantı sayısını `static` bir sayaçta tutsun. Ardından iki sürücü program yazınız: biri `try-with-resources` kullansın, diğeri kaynağı elle açıp istisna fırlatsın. İki programın sonunda açık bağlantı sayısını yazdırarak sızıntıyı **sayısal olarak** gösteriniz.

İpucu: Föydeki Görev 4 (`GunlukDosyasi`) tam olarak bu sayaç düzenini kullanır; `acilanSayisi()` ve `kapananSayisi()` metotlarını örnek alınız.

11. Öz Değerlendirme

Aşağıdaki maddelerin tamamını işaretleyebiliyorsanız bu haftanın kazanımlarına ulaşmışsınız demektir. İşaretleyemediğiniz her madde için karşısındaki bölüme dönünüz.

Öz Değerlendirme Kontrol Listesi

- ☐ `new` ifadesinin beş adımını sayabiliyor ve yapıcının hangi adım olduğunu söyleyebiliyorum. *(Bölüm 3.1, Şekil 5.2)*
- ☐ Yapıcı aşırı yükleme kuralını uygulayabiliyor, hangi ikilinin derlenmeyeceğini önceden söyleyebiliyorum. *(Bölüm 3.3)*
- ☐ `this(...)` zinciri kurabiliyor, gövdelerin hangi sırayla çalışacağını çıktıya bakmadan yazabiliyorum. *(Bölüm 3.4)*
- ☐ Doğrulamayı tek bir ana yapıcıda toplayabiliyorum; kuralın iki kez yazıldığı bir kodu tanıyabiliyorum. *(Bölüm 2.2, 3.5)*
- ☐ Kopya yapıcı yazabiliyor, hangi alanın kopyalanması gerektiğine gerekçeli karar verebiliyorum. *(Bölüm 4.1)*
- ☐ Sığ ve derin kopya farkını somut bir çıktıyla açıklayabiliyorum. *(Bölüm 4.2, Şekil 5.5)*
- ☐ Statik fabrika metodu yazabiliyor, dört üstünlüğünü ve bedelini sayabiliyorum. *(Bölüm 4.3)*
- ☐ Nesne başlatma sırasını doğru sırayla yazabiliyorum. *(Bölüm 4.4)*
- ☐ Java'da yıkıcı bulunmamasının nedenini ve `finalize()`'ın neden kullanılmadığını açıklayabiliyorum. *(Bölüm 4.5)*
- ☐ `AutoCloseable` gerçekleyen bir sınıf yazıp `try-with-resources` ile kullanabiliyorum. *(Bölüm 4.6)*

12. Kaynaklar

12.1. Ders Kitapları

Kaynak	İlgili bölüm	Not
Deitel & Deitel, *Java How to Program* (11. baskı)	Bölüm 8.4–8.5 — Overloaded Constructors, `this` Reference; Bölüm 8.10 — Garbage Collection	Temel düzey, bol örnekli
Horstmann, *Core Java Volume I* (12. baskı)	Bölüm 4.6 — Object Construction (overloading, default field initialization, initialization blocks)	Başlatma sırası için en açıklayıcı anlatım
Schildt, *Java: The Complete Reference* (12. baskı)	Bölüm 6 — Constructors, `this`, GC; Bölüm 13 — try-with-resources	Başvuru kaynağı
Bloch, *Effective Java* (3. baskı)	Madde 1, 2, 8, 9 — fabrika metodu, Builder, `finalize`, try-with-resources	Haftanın tasarım felsefesi

Tablo 5.17 — Bu hafta için önerilen okumalar

12.2. Çevrimiçi Kaynaklar

- **Oracle Java Tutorials — Providing Constructors for Your Classes:** yapıcı yazımı ve varsayılan yapıcı kuralı.
- **Oracle Java Tutorials — Initializing Fields:** örnek/statik başlatma blokları ve başlatma sırası.
- **Oracle Java Tutorials — The try-with-resources Statement:** `AutoCloseable`, bastırılmış istisnalar ve kapanma sırası.
- **Java Language Specification, §8.8.7.1 — Explicit Constructor Invocations:** `this(...)` çağrısının biçimsel kuralları (ileri düzey okuma).
- **JEP 421 — Deprecate Finalization for Removal:** `finalize()` metodunun neden kaldırıldığının resmî gerekçesi.

12.3. İletişim ve Destek

Konu	Sorumlu	Kanal
Teori, kavram soruları	Dr. Öğr. Üyesi Rıfat BENVENİSTE	Ders saati sonrası · e-posta
Laboratuvar, kod hataları	Arş. Gör. Betül MUMCU	Laboratuvar saatleri · e-posta
Teslim ve puanlama	Arş. Gör. Betül MUMCU	Ders yönetim sistemi
Ortam kurulumu (JDK, VS Code)	Laboratuvar sorumlusu	Hafta 1 föyü, Görev 0

Tablo 5.18 — Soru ve destek için başvuru noktaları

> **ⓘ BİLGİ**
>
> Bu haftanın tüm Java kaynak dosyaları `Hafta_05/Java_Kodlari/` klasöründedir: ders örnekleri `ders/`, laboratuvar çözümleri ise laboratuvar oturumundan sonra `laboratuvar/` altında paylaşılacaktır. Kod bloklarındaki satır numaraları bu dosyalarla birebir örtüşür.